INSTITUTO FEDERAL
Goiás

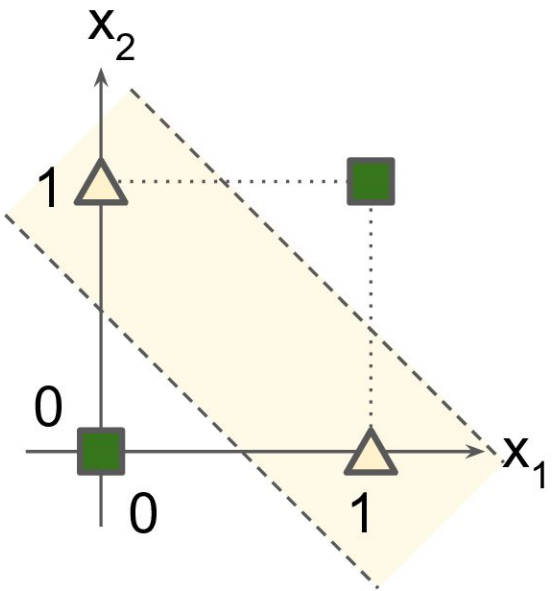
Deep Learning

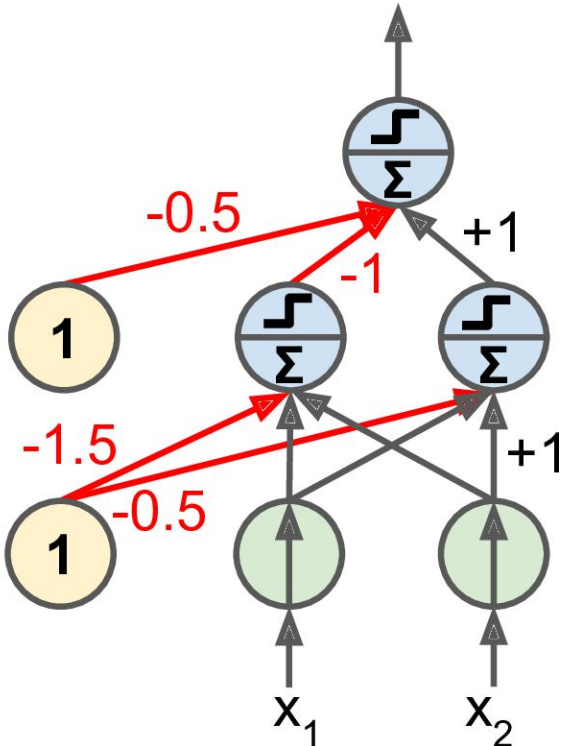
ML Perceptron, Backpropagation & Exemplos

Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

Sabemos que…

- Ao incluir ao menos uma camada escondida (hidden layer)...
  - Resolvemos o XOR





Mas funciona? Isto pode dar certo?

Teorema de Kolmogorov

- Em 1900, o matemático David Hilbert formulou 23 problemas desafiadores em que os matemáticos do século XX deveriam trabalhar.
- Em 1956, um desses problemas foi parcialmente resolvido por Kolmogorov e seu estudante Arnol'd
- Melhorias no teorema de Kolmogorov foram obtidas por Sprecher (1963) e Lorentz (1966).

Theorem 1 (Kolmogorov's Superposition Theorem). *For each $n \geq 2$ there exist continuous functions $\varphi_q : [0,1] \to \mathbb{R}$, $q = 0, ..., 2n$ and positive constants $\lambda_p \in \mathbb{R}$, $p = 1, ..., n$ such that the following holds true: for each continuous function $f : [0,1]^n \to \mathbb{R}$ there exists a continuous function $g : [0,1] \to \mathbb{R}$ such that*

$$f(x_1, ..., x_n) = \sum_{q=0}^{2n} g\left(\sum_{p=1}^{n} \lambda_p \varphi_q(x_p)\right).$$

Teorema de Kolmogorov - 2

Theorem 1 (Kolmogorov's Superposition Theorem). *For each $n \geq 2$ there exist continuous functions $\varphi_q : [0,1] \to \mathbb{R}$, $q = 0, ..., 2n$ and positive constants $\lambda_p \in \mathbb{R}$, $p = 1, ..., n$ such that the following holds true: for each continuous function $f : [0,1]^n \to \mathbb{R}$ there exists a continuous function $g : [0,1] \to \mathbb{R}$ such that*

$$f(x_1, ..., x_n) = \sum_{q=0}^{2n} g\left(\sum_{p=1}^{n} \lambda_p \varphi_q(x_p)\right).$$

- Exemplo com n=2: Existem cinco funções de uma única variável $\varphi_0, \varphi_1, \cdots, \varphi_4$ e constantes λ_1 e λ_2 tais que <u>toda</u> função contínua f(x,y) pode ser escrita <u>de forma exata</u> como

$$f(x,y) = g(\lambda_1\varphi_0(x) + \lambda_2\varphi_0(y)) + g(\lambda_1\varphi_1(x) + \lambda_2\varphi_1(y)) + \ldots + g(\lambda_1\varphi_4(x) + \lambda_2\varphi_4(y))$$

Teorema de Kolmogorov - 3

- Cinco funções de uma única variável → para toda e qualquer f(x,y)
- Constantes λ_1 e λ_2 → para toda e qualquer f(x,y)
- Com as $\varphi_0, \varphi_1, \ldots, \varphi_4$ crie cinco features baseadas apenas em x e apenas em y:
  - $\varphi_0(x), \varphi_1(x)\varphi_2(x), \varphi_3(x), \varphi_4(x)$
    $\varphi_0(y), \varphi_1(y)\varphi_2(y), \varphi_3(y), \varphi_4(y)$
- Imagine, por exemplo, que $\varphi_0(x) = x, \varphi_1(x) = x^2, \ldots, \varphi_4(x) = x^5$
- Forme combinações lineares duas a duas das cinco features (a MESMA combinação)
- Use uma função de ativação g e some os resultados: obtemos f(x,y)

$$f(x,y) = g(\lambda_1\varphi_0(x) + \lambda_2\varphi_0(y)) + g(\lambda_1\varphi_1(x) + \lambda_2\varphi_1(y)) + \ldots + g(\lambda_1\varphi_4(x) + \lambda_2\varphi_4(y))$$

Teorema de Kolmogorov e redes neurais

INSTITUTO FEDERAL
Goiás

- Em 1987, Hecht-Nielsen aplica o Kolmogorov's Superposition Theorem para mostrar que toda função contínua de n inputs pode ser escrita como uma rede neural com apenas uma camada oculta (com 2n+1 neurônios e a função de ativação proposta por eles).

Hecht-Nielsen (1987) Proceedings IEEE Int Conf Neural Networks, vol II

Relevância do teorema de Kolmogorov

INSTITUTO FEDERAL
Goiás

- A prova do teorema não é construtiva: não nos diz como obter as funções envolvidas
- É um teorema de existência.
- Ela nos diz que essa rede neural de uma camada oculta deve existir, mas não nos diz como encontrá-la.
- Recentemente foi desenvolvida uma rede que explora o teorema:
    - https://github.com/KindXiaoming/pykan (2025) - redes mais interpretáveis, bons regressores de funções (retorna a fórmula da função).

- Traz segurança e suporte à área de deep learning, pois…
    - redes neurais de 1 camada oculta são capazes de representar toda e qualquer função de n inputs.
    - Isto dá a “esperança” de que nossas redes com ativações como sigmoid e relu consigam representar bem as funções funções arbitrárias (mas Kolmogorov não provou isso)...

Teorema de Kolmogorov e redes neurais

- Avançando o teorema de Kolmogorov:
  - Com funções de ativação simples (ex. sigmoid) o argumento é suficiente, mas o número de neurônios da camada oculta pode ser muito elevado (Huang 2003).
  - Usar uma segunda camada oculta drasticamente reduz o número total de neurônios (Stathkis 2008).
  - Hornik, Stinchombe e White (1989), teorema da aproximação universal

D. Stathkis (2008). How many hidden layers and nodes? Jour. Remote Sensing, vol 30

Neural Networks, Vol. 2, pp. 359–366, 1989
Printed in the USA. All rights reserved.

0893-6080/89 $3.00 + .00
Copyright © 1989 Pergamon Press plc

ORIGINAL CONTRIBUTION

Multilayer Feedforward Networks are Universal Approximators

KURT HORNIK

Technische Universität Wien

MAXWELL STINCHCOMBE AND HALBERT WHITE

University of California, San Diego

(*Received* 16 *September* 1988; *revised and accepted* 9 *March* 1989)

Teorema da aproximação universal

INSTITUTO FEDERAL
Goiás

- Hornik, Stinchombe e White (1989) provaram um teorema muito importante
- Conhecido como Teorema da Aproximação Universal

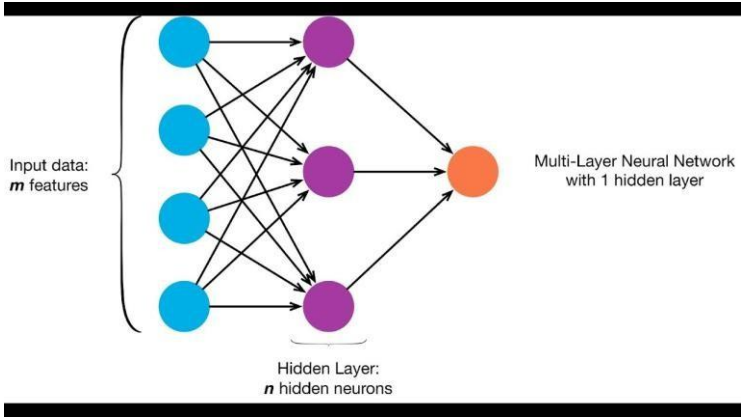
- **Abstract:**
- This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer (...) are capable of approximating (...) a function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. **In this sense, multilayer feedforward networks are a class of universal approximators.**

Teorema da aproximação universal - debulhando...

- Qualquer problema de classificação ou regressão pode ser resolvido com uma rede neural com uma única camada.
- Talvez a camada precise de muitos neurônios mas basta uma única camada!!
- Seja h(x) uma função dsconhecida:

  Existe uma rede neural com uma única camada tal que sua saída $\hat{y}(\mathbf{x})$ é muito próxima de h(x) para todo x
- Escolha ϵ bem pequeno (por exemplo,) $\epsilon = 10^{-5}$
- Então existe rede neural de uma camada tal que

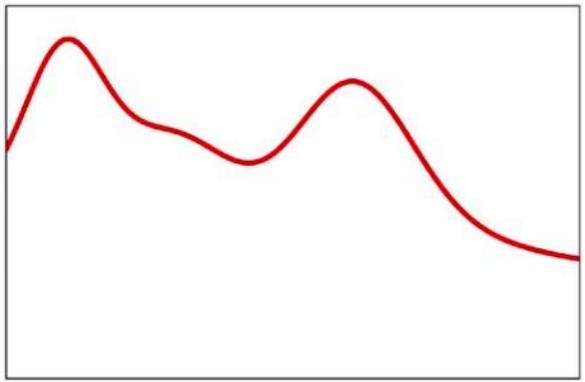
$$|h(\mathbf{x}) - \hat{y}(\mathbf{x})| < \epsilon \quad \forall \mathbf{x}$$



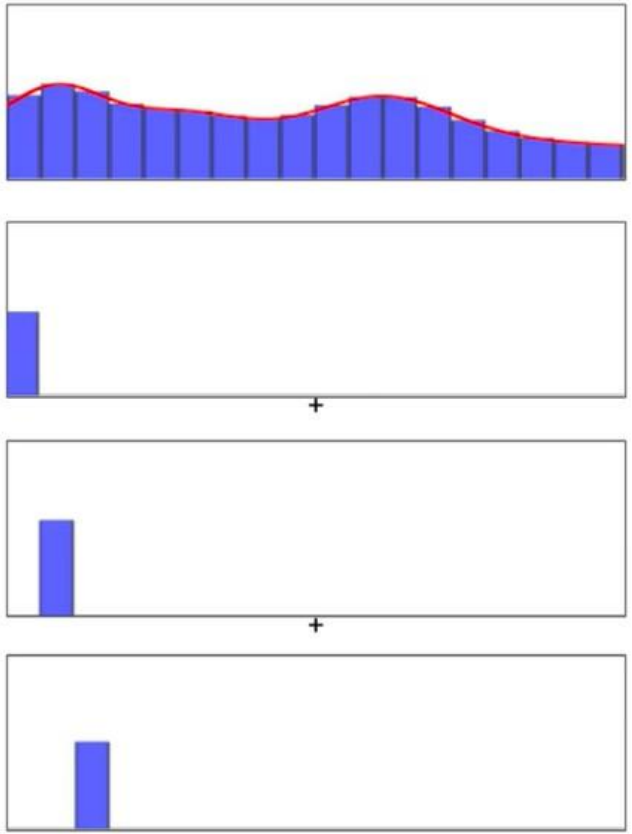
Illustrative Proof of Universal Approximation Theorem

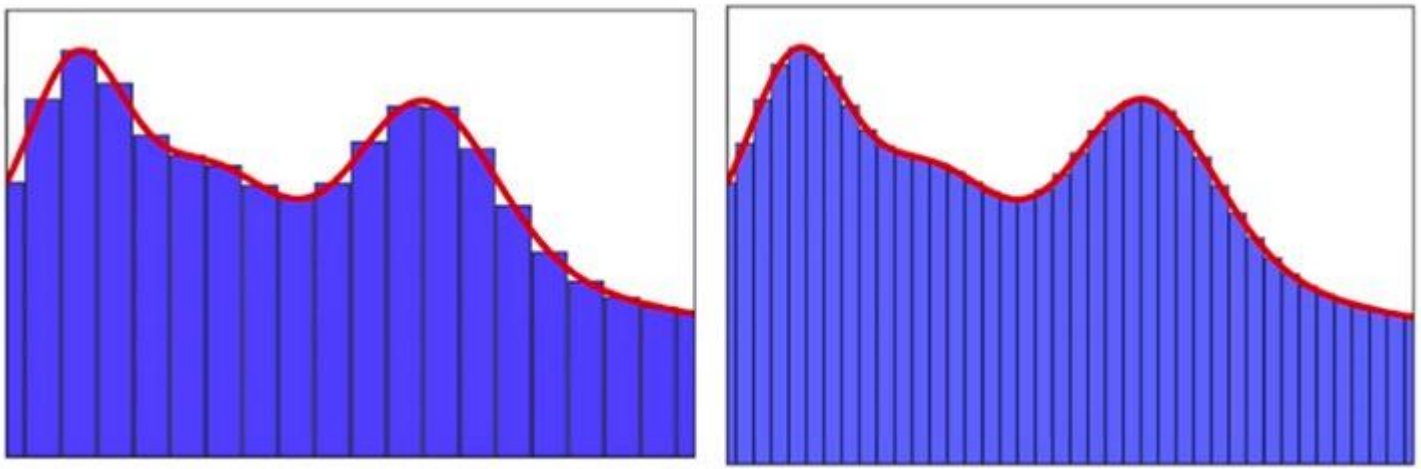
INSTITUTO FEDERAL
Goiás

- Extraído de post de Niranjan Kumar
  - https://hackernoon.com/illustrative-proof-of-universal-approximation-theorem-5845c02822f6
- Suponha que seja a função abaixo
- Nós não conhecemos (nem vemos) a “fórmula” h(x)
- Não temos uma representação h(x)



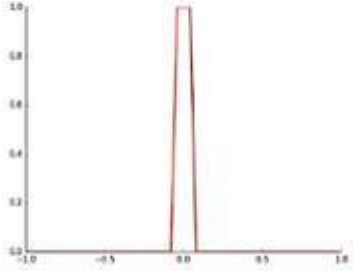
- Aproximamos a função h(x) por funções escada (step functions)
- Cada step function é função diferente de zero apenas num dos intervalos-bins
- h(x) é aprox a soma das step functions

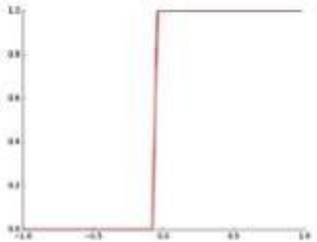


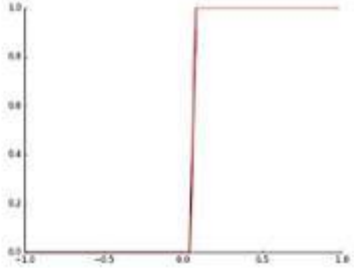


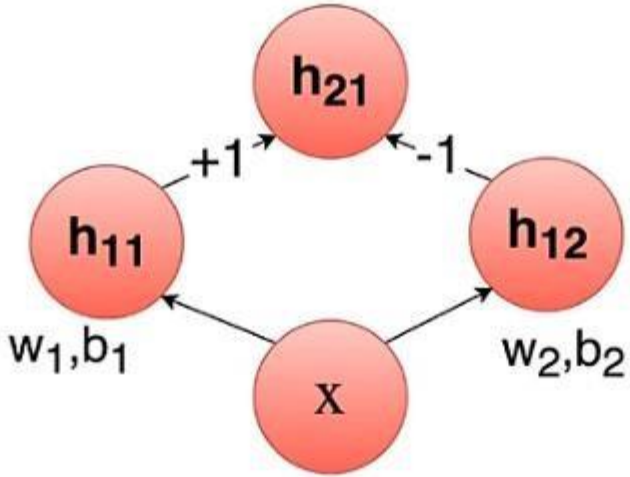
- Podemos fazer a aproximação ser tão boa quanto quisermos (menor que qualquer epsilon fixado)
- Basta tomar bins-intervalos cada vez menores
- Mas como estas step functions são geradas?

INSTITUTO FEDERAL
Goiás









Relevância do teorema universal

INSTITUTO FEDERAL
Goiás

- O teorema da aproximação universal é um teorema poderoso.
- Ele indica que uma rede neural é um aproximador universal de funções: qualquer função pode ser extremamente bem aproximada por uma rede neural de UMA camada escondida.
- No entanto, ele não diz nada sobre o tamanho da rede neural de camada oculta.
- Apenas que é finita.
- E que você pode tornar a rede neural arbitrariamente próxima da função original simplesmente lançando mais unidades ocultas nela.
- Serve como uma base de sustentação de que redes neurais dão conta do recado.
- IMPORTANTE: rede simples de duas camadas pode ser muito melhores que rede complexa de uma única camada. Ver exemplo a seguir.

Modelo gerador dos dados é uma árvore com dois inputs

- Rede com 6 unidades em uma camada escondida faz um serviço ok.

- Precisa de muito mais unidades com uma camada.

- Rede de duas camadas com 3 + 4 unidades é perfeita.

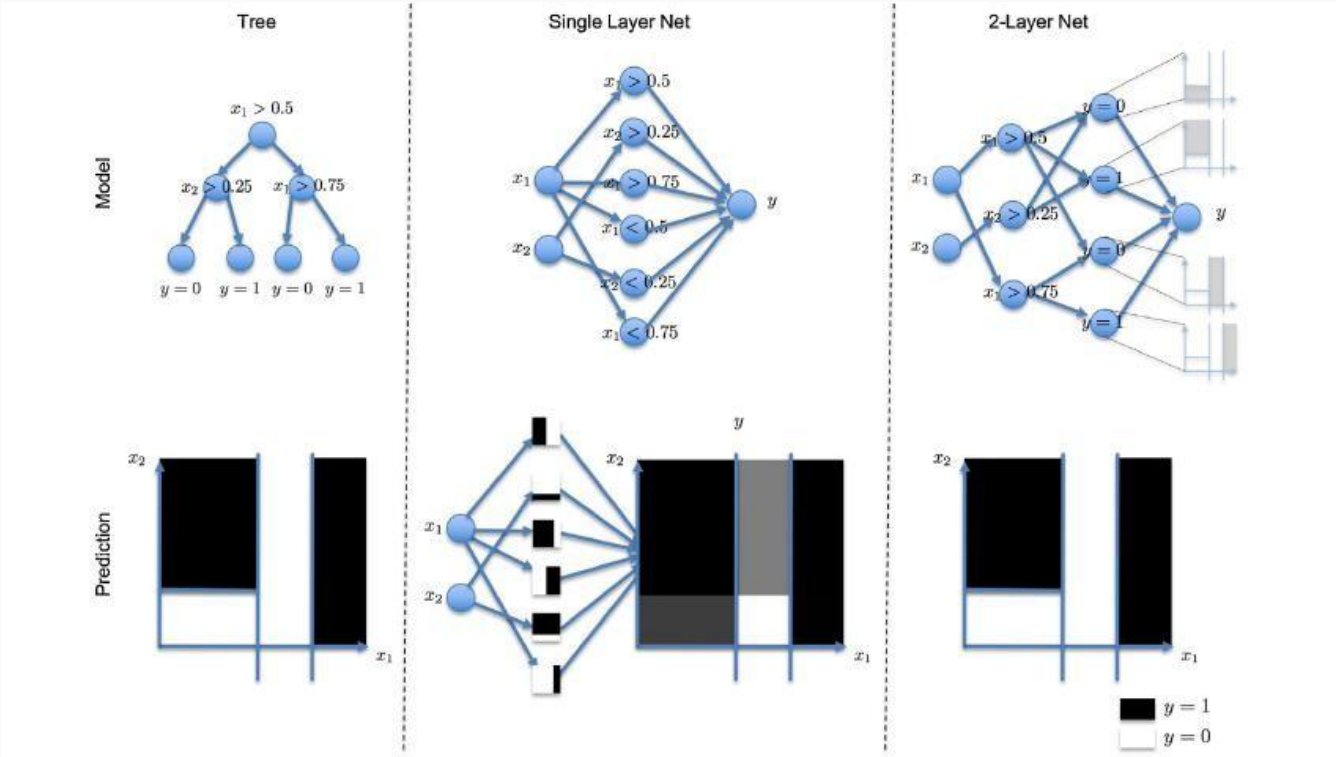


Figure 3. A decision tree allows to describe any partition of space and can thus model any decision boundary. Mapping the tree into a one-layer network is possible. Yet, there still is significant residual error in the resulting function. In the center example, $\epsilon \approx 0.7$. In order to reduce this error further, a higher number of neurons would be required. If we construct a network with one node for every inner node in the first layer and one node for every leaf node in the second layer, we are able to construct a network that results in $\epsilon = 0$.

From Maier et al (2019)A gentle introduction to deep learning in medical image processing. Z. Med. Phys 29, 86-101

Como escolher o tamanho de uma camada escondida?

INSTITUTO FEDERAL
Goiás

- Podemos escolher a dimensionalidade (o número de nós) da camada oculta.
- Quanto mais nós colocarmos na camada oculta, mais complexas podem ser as funções aproximadas.

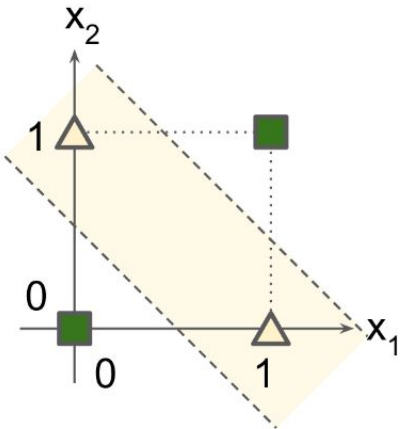
- Mas a maior dimensionalidade tem um custo.
  - Primeiro, mais computação é necessária para fazer previsões e aprender os parâmetros da rede.
  - Um número maior de parâmetros também significa que nos tornamos mais propensos a sebreajustar aos nossos dados.
- Para construção da arquitetura, é necessária muita experimentação e análise guiada pela interpretação dos resultados.

Funções de Ativação

- O que já sabemos sobre funções de ativação e multilayer perceptrons?

Funções de Ativação

- O que já sabemos sobre funções de ativação e multilayer perceptrons?
  - Podemos criar funções de ativação complexas (i.e. função de kernel) para tratar problemas complexos… mas encontrar essas funções é difícil! (e não é prático)
  - Uma função de ativação simples (step function) com uma multilayer perceptron com camada oculta é um aproximador universal!!
    - ex.: traçando múltiplos hiperplanos…

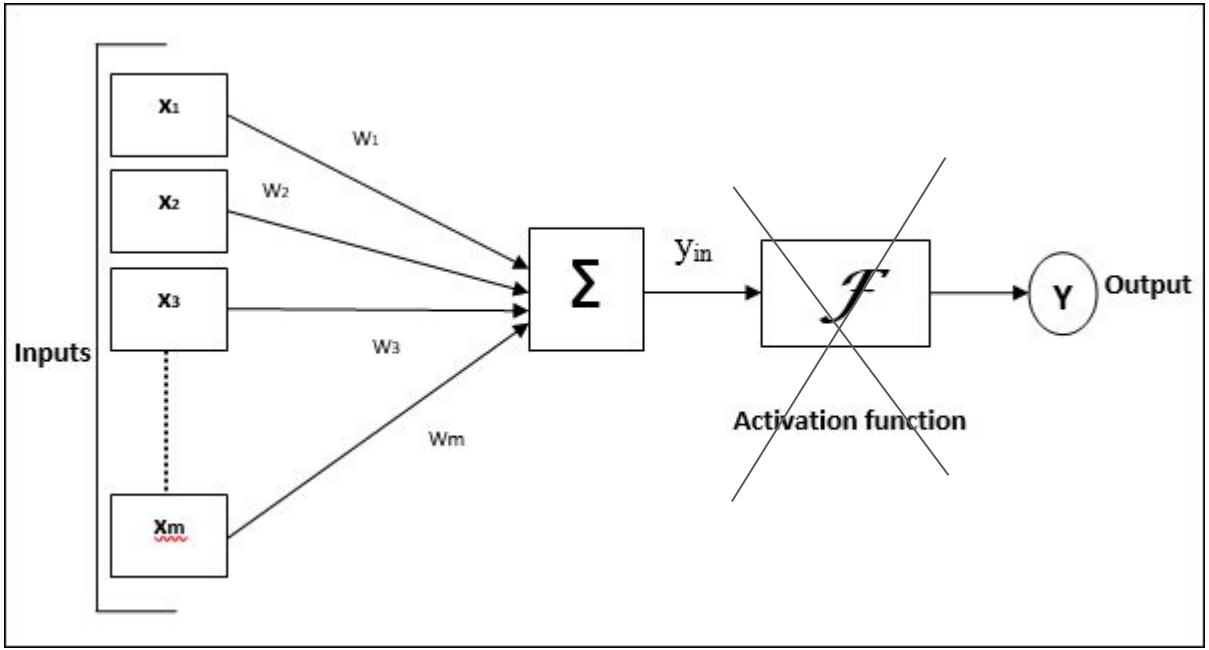


Funções de Ativação

- O que já sabemos sobre funções de ativação e multilayer perceptrons?
  - Podemos criar funções de ativação complexas (i.e. função de kernel) para tratar problemas complexos… mas encontrar essas funções é difícil! (e não é prático)
  - Uma função de ativação simples (step function) com uma multilayer perceptron (várias unidades perceptron) com camada oculta pode resolver problemas difíceis.
    - traçando múltiplos hiperplanos…
- **Mas será que se usarmos múltiplos neurônios e camadas ocultas, precisamos mesmo de função de ativação?**

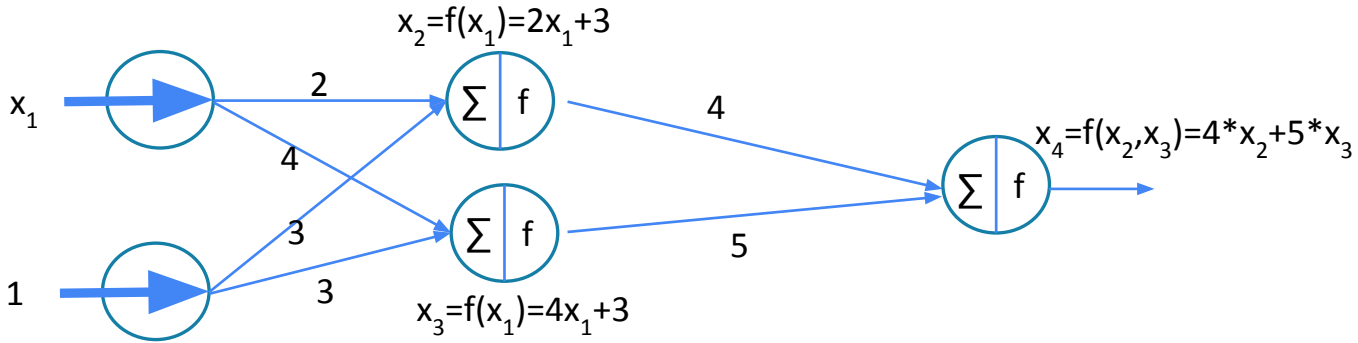
Funções de Ativação

- Suponha que não temos mais a função de ativação nas camadas ocultas.
  - outro sinônimo… função de ativação “linear”.



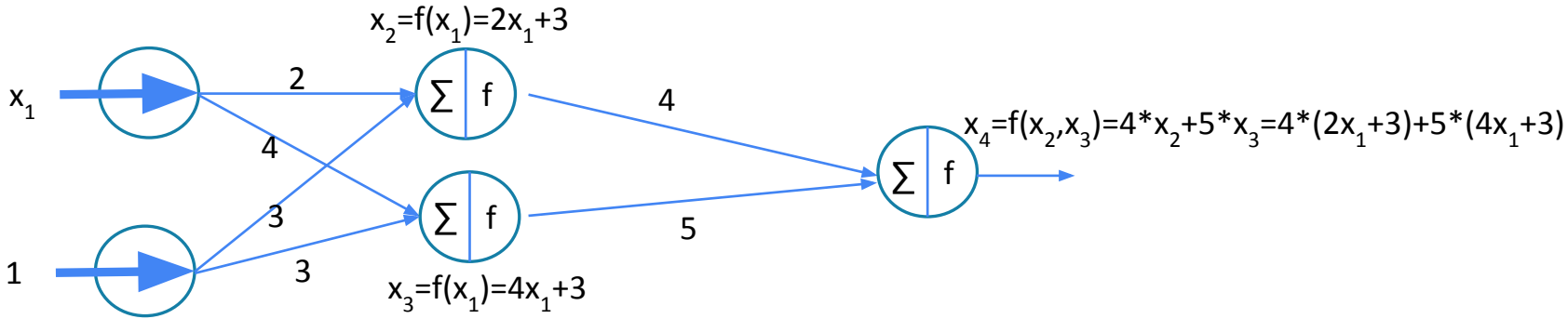
Função de Ativação Linear

- O que acontece com a função de ativação linear?



Função de Ativação Linear

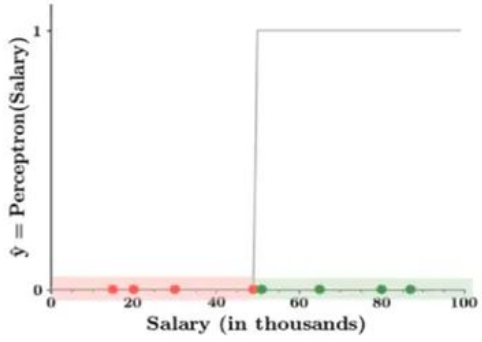
- O que acontece com a função de ativação linear?
  - composição de funções lineares gera outras funções lineares.
    - Portanto… precisamos de uma função de ativação que não seja linear se não queremos um único hiperplano separador como saída de cada neurônio!

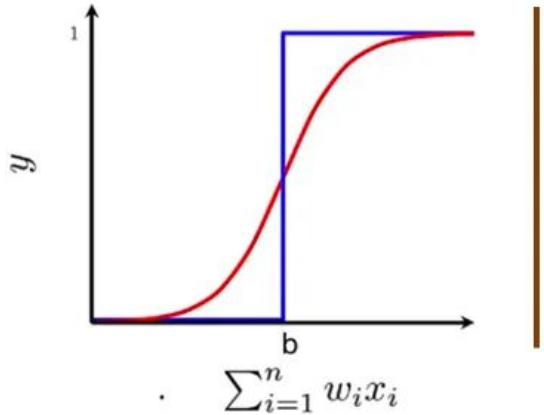


Função de Ativação Sigmoid

- Uma das primeiras funções (não step) a serem usadas em multilayer perceptrons.
  - Neurônios também chamados de “sigmoid neurons”
- Sempre produz valores entre 0 e 1
- “bioinspirada” - Resposta próxima a de neurônios biológicos.
- Diferenciável!
- Usada em tarefas de classificação e regressão.
- Neurônios não produzem saídas “draconianas”.

Salary (in thousands)	Can buy a car?
80	1
20	0
65	1
15	0
30	0
49	0
51	1
87	1



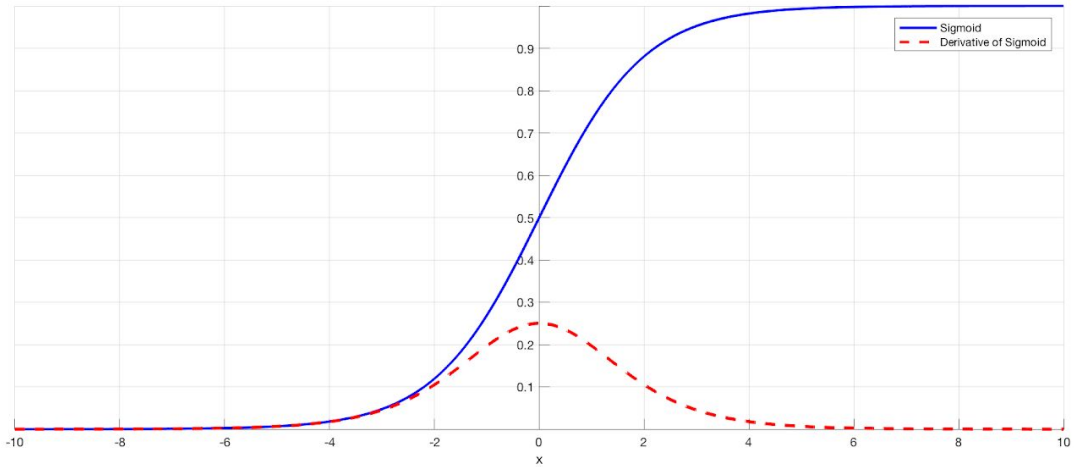


$$y = \frac{1}{1+e^{-(w^T x+b)}}$$

Função de Ativação Sigmoid

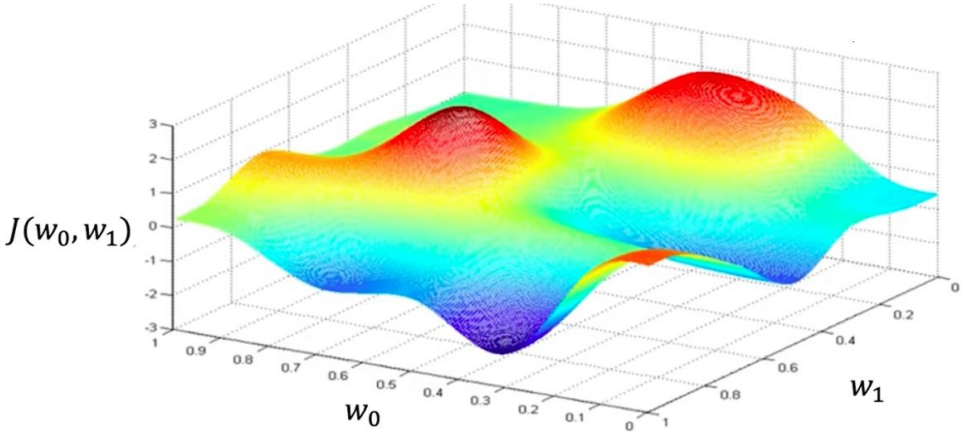
- Diferenciável

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$



Função de Ativação Sigmoid

- Diferenciável, portanto podemos usar o gradiente para ajustar pesos…
  - Intuição: Para o erro (custo) MSE=J(w1,w2)



$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

makes result quadratic

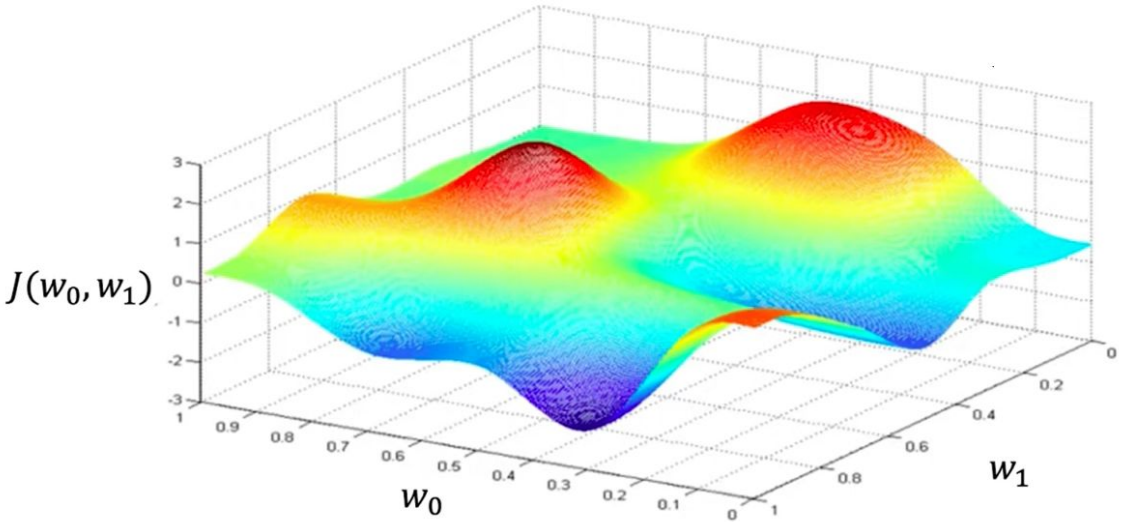
y_i: true y

$\hat{y}_i$: estimate of y

$\frac{1}{N}$: average over all results

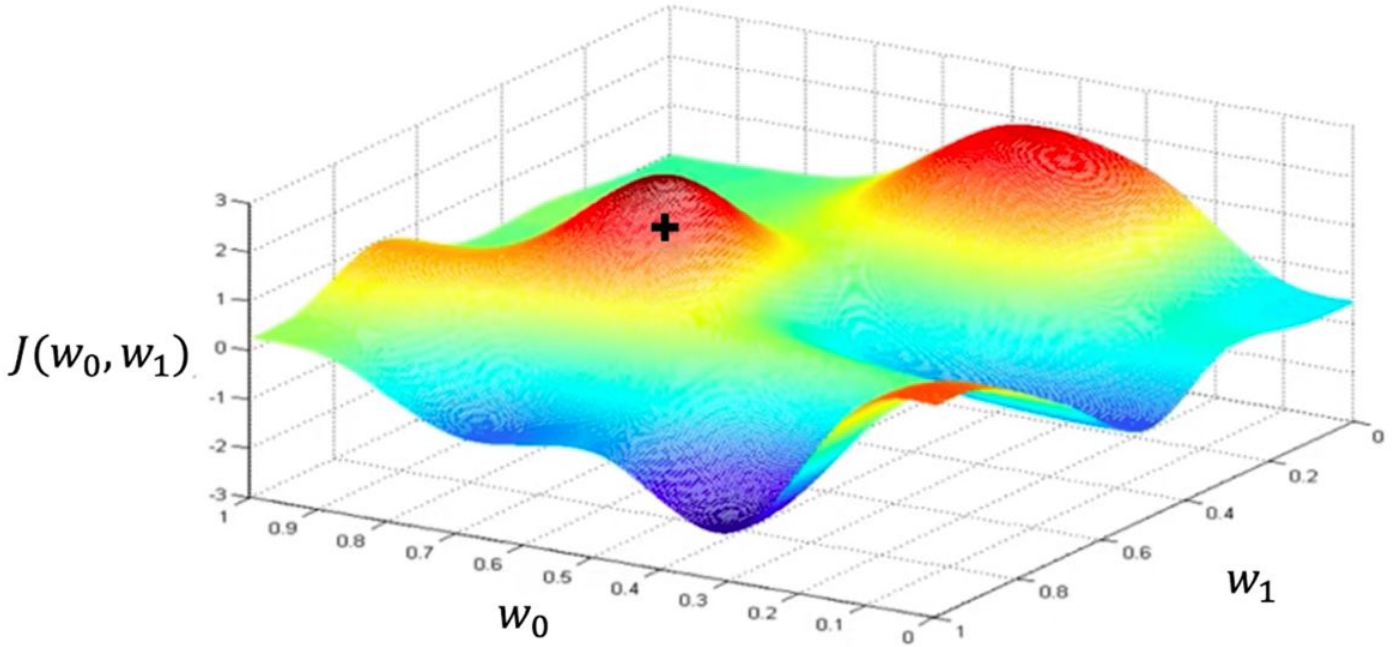
Gradiente - Intuição

- Considerando uma rede com apenas dois pesos, podemos visualizar a função de perda da rede considerando esses dois pesos.
  - Queremos os pontos com menor perda!



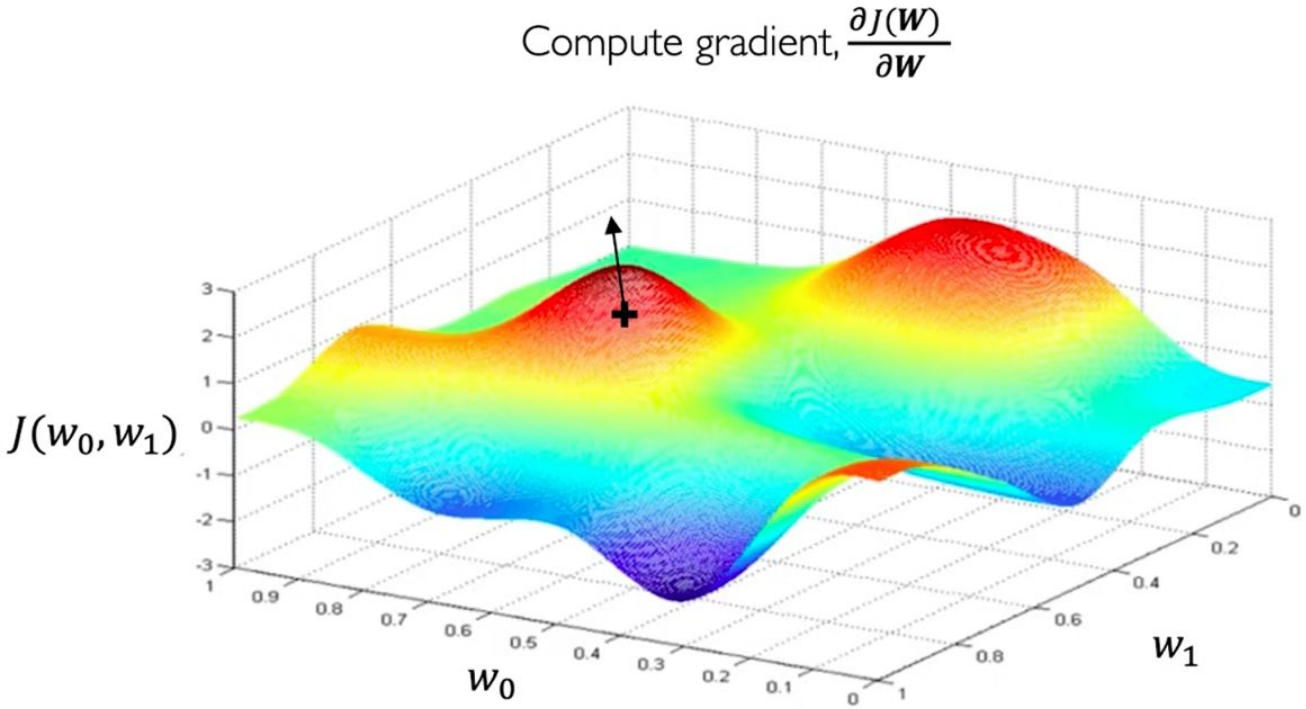
Gradiente - Intuição

- Iniciamos nossa busca pegando um ponto aleatório



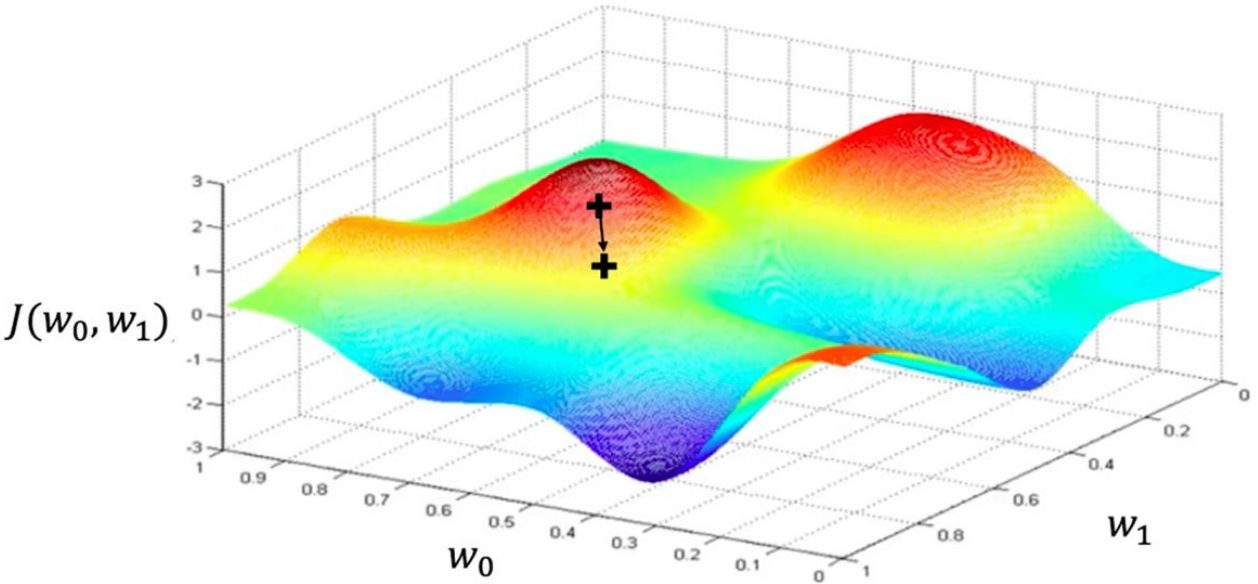
Gradiente - Intuição

- Depois, computamos o gradiente no ponto escolhido
  - ele nos dá a direção que maximiza J(w0, w1)



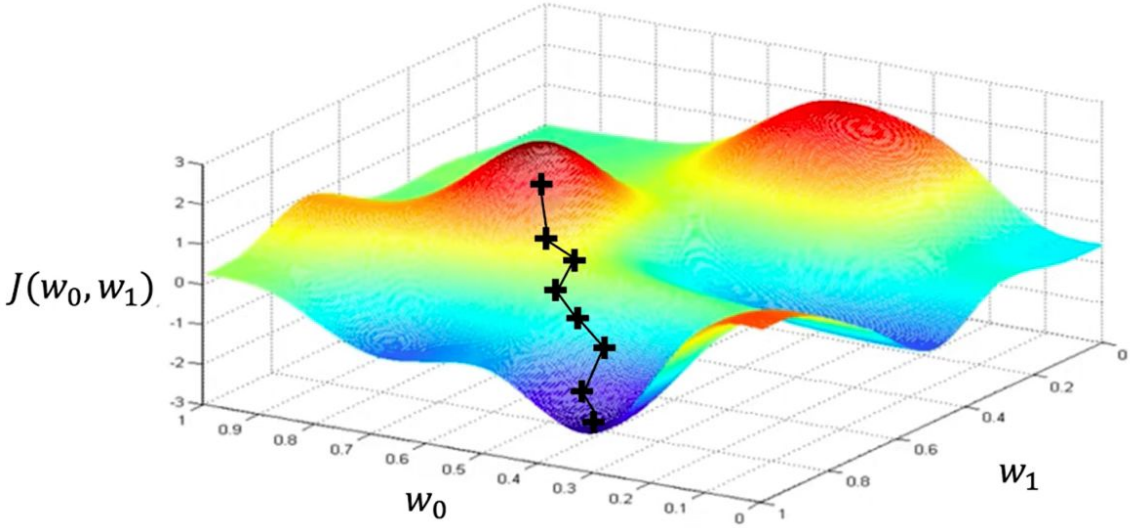
Gradiente - Intuição

- Inverte a direção (negativo do gradiente)
  - toma um pequeno passo em relação à direção descoberta.



Gradiente - Intuição

- Repita o processo até a convergência
  - Mínimo local



Incluindo o gradiente no nosso exemplo

Predição(w, x, bias)

ativação = sigmoid(w^T * x + bias)

retorna ativação

Repita (até convergência dos pesos)

Para cada instância (x,y)

bias=bias - α *gradiente_bias

Para cada peso w_k

w[k] = **w**[k] - α**gradiente*_x[k]

Bibliotecas



Livros:

1 François Chollet. Deep Learning with Python. Manning - Author do Keras!

2 Aurélien Géron. Neural networks and deep learning https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html - exemplos em tensorflow

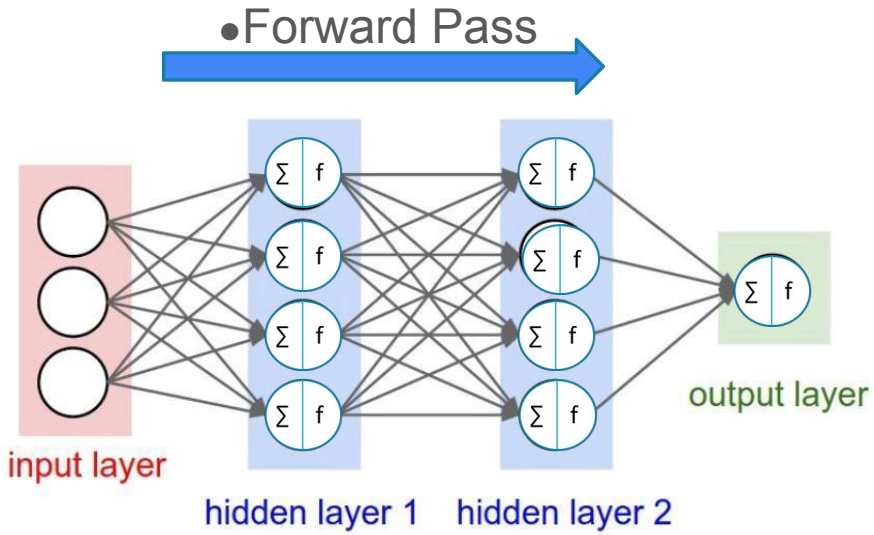
Autoinscrição - Moodle

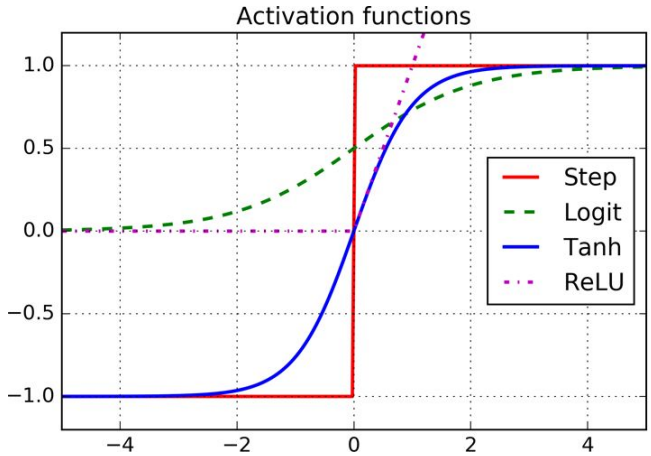
https://moodle.ifg.edu.br/course/view.php?id=59560

Senha: dl20262

Noção Geral do MLP

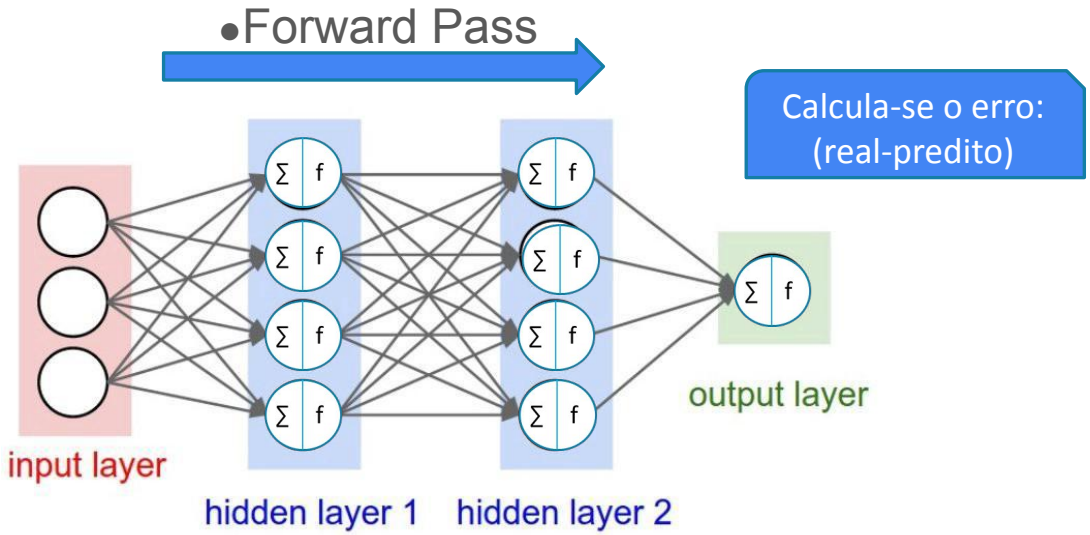
- Considere uma parte do treino (mini-batch), 32 instâncias por exemplo.
- Substituição da *step function* por *sigmoid function*.





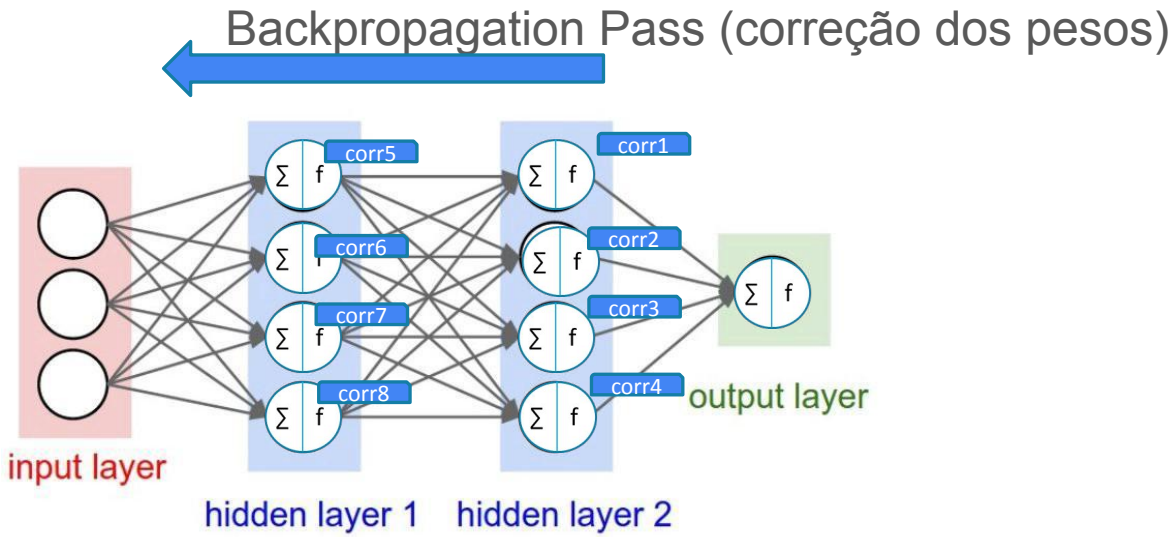
Noção Geral do MLP

- Considere uma parte do treino (mini-batch), 32 instâncias por exemplo.



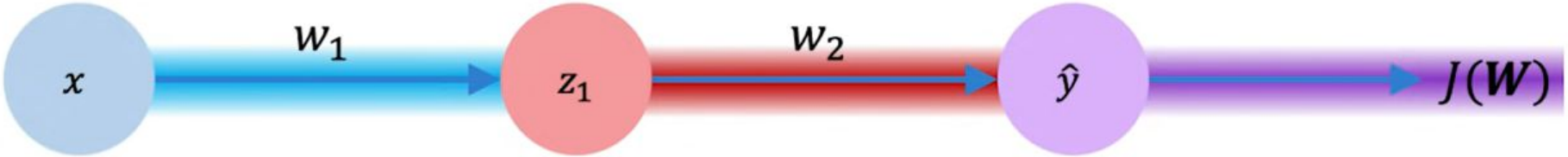
Noção Geral do MLP

- Considere uma parte do treino (mini-batch), 32 instâncias por exemplo.



Backpropagation - Gradiente

- **Algoritmo**



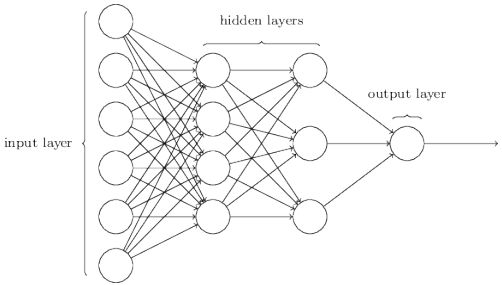
$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

$$\frac{\partial J(\boldsymbol{W})}{\partial w_1} = \frac{\partial J(\boldsymbol{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

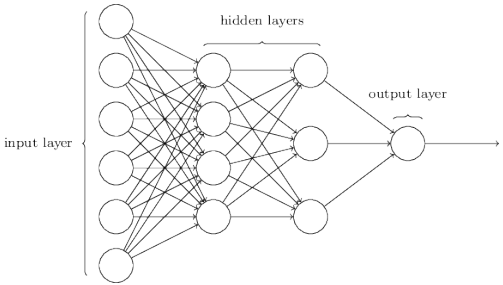
Tudo na sua época...

- Em cada epoch
  - passada por toda coleção de treinamento
    - A atualização dos pesos pode ser dividida em subconjuntos (batches) em cada época.

Epoch 1



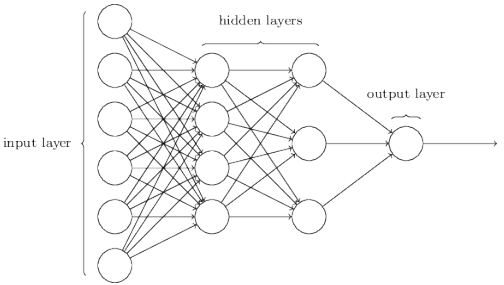
Epoch 2



Epoch ...

...

Epoch 3



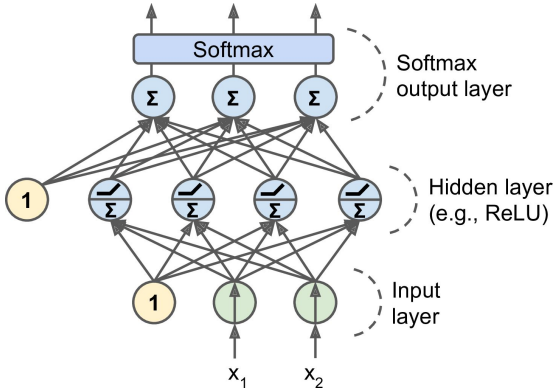
MLP: Regressão

- Para modelos de regressão, desejando um único valor, basta um neurônio de saída;
- De forma geral, se aplica um neurônio por dimensão:
  - Exemplo: para colocalizar um ponto em uma imagem, a rede deve ter dois neurônios de saída.
- Em regressão, frequentemente não se usam função de ativação no neurônio de saída. Contudo, algumas funções podem garantir o intervalo de valores, como a ReLu ou Logistic;
- Segue uma arquitetura típica para regressão:

Hiperparâmetros	Valores Típicos
# neurônios de entrada	Um por atributo
# camadas ocultas	Depende do problema, mas tipicamente entre 1 e 5
# neurônios por camada oculta	Depende do problema, mas tipicamente entre 10 e 100
# neurônios de saída	Um por dimensão de predição
Função de ativação oculta	ReLu
Função de ativação de saída	Nenhum, ReLu ou Logistic
Loss Fuction	MSE ou MAE

MLP: Classificação

- Para classificação binária, um único neurônio de saída pode ser suficiente.
  - Usando uma função de ativação logística;
  - Permitindo a interpretação de uma função de probabilidade de classe positiva.
- Para problemas multi-classe, se aplica uma um neurônio de saída para cada classe;
- Aplicando uma camada softmax, garantindo a distribuição de probabilidade entre as classes.



MLP: Classificação

- Segue uma arquitetura típica para classificação:

Hiperparâmetros	Class. Binária	Multi Classificação
Entrada e camadas ocultas	Mesmo que regressão	Mesmo que regressão
# neurônios de saída	Um	Um por classe
Função de ativação de saída	Logistic	Softmax
Loss Fuction	Cross Entropy	Cross Entropy

INSTITUTO FEDERAL
Goiás

Atividades (em aula)

Utilize o TensorFlow Playground: http://playground.tensorflow.org , use este simulador de redes neurais para avaliar diferentes MLPs com entradas X_1 e X_2

Caso 1) Simule uma perceptron (sem camadas ocultas) nas tarefas de “ou exclusivo” e “gaussiana” para a tarefa de classificação. Descreva os resultados obtidos em função da resolução dos problemas.

Caso 2) Inclua uma camada oculta com 6 neurônios e verifique os resultados do problema “ou exclusivo” em diferentes funções de ativação. Houve alguma função de ativação que não possível resolver o problema, mesmo com a camada oculta? Porque?

Caso 3) Avalie uma tarefa de classificação, usando dados com distribuição gaussiana e com 30% dos dados para teste.

- Como é possível melhorar o aprendizado para padrões mais complexos?
- Quais camadas aprendem padrões mais complexos: lower layers ou upper layer?

Caso 4) Defina uma rede com dados do tipo círculo e uma camada interna com dois neurônios. Rode até 30 epochs. Avalia os erros de teste e treino. Execute novamente a rede, mas com três camadas ocultas e quatro neurônio em cada. Registre suas conclusões.

Caso 5) Utilize o tipo de dado em espiral e rode uma rede com três camadas ocultas e quatro neurônio em cada. Após 100 epochs o que é possível observar?

Caso 6) Utilize o tipo de dado em espiral e rode com apenas uma camada oculta considerando 3 neurônios e 8 neurônios. O que é possível observar em relação ao Caso 5?

Referências

[1] Aurélien Géron. Neural networks and deep learning (cap. 1) https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html

[2] NIELSEN, M. A. Neural Networks and Deep Learning. Determination Press, 2015. [Online]. Disponível em <http://neuralnetworksanddeeplearning.com/>

[3] GOODFELLOW, Y. BENGIO, A. COURVILLE. Deep Learning. MIT Press, 2017.

BackPropagation

- Criado em 1970, pelo estudante de mestrado finlandês Seppo Linnainmaa.
  - Porque pouca gente valorizou na época?
    - “Taylor expansion of the accumulated rounding er

ALGORITMIN KUMULATIIVINEN PYÖRISTYSVIRHE

YKSITTÄISTEN PYÖRISTYSVIRHEIDEN TAYLOR-KEHITELMÄNÄ

- Realmente valorizado (“redescoberto”) em 1986, no paper famoso de Rumelhart, Geoffrey Hinton e Ronald Williams
- Backpropagation versa sobre entender como iremos mudar os pesos e o bias, seguindo as mudanças na função custo.
- Logo, requisitos importantes para função de custo em redes neurais:
  - A função custo deve ser obtida a partir de uma média na análise de perda individual.
  - A função custo deve ser escrita como uma função de saída da rede neural
- Uma das mais populares idéias em IA
  - >25 mil citações!

Learning representations by back-propagating errors

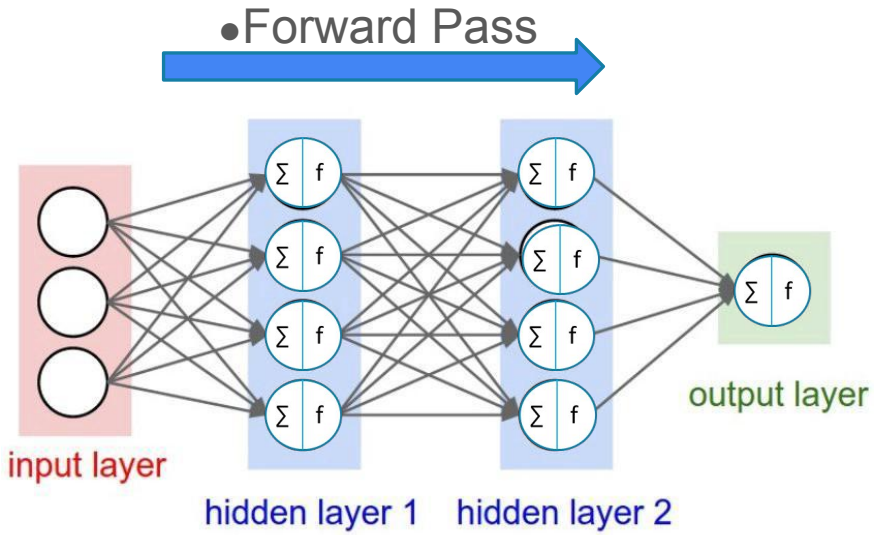
David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

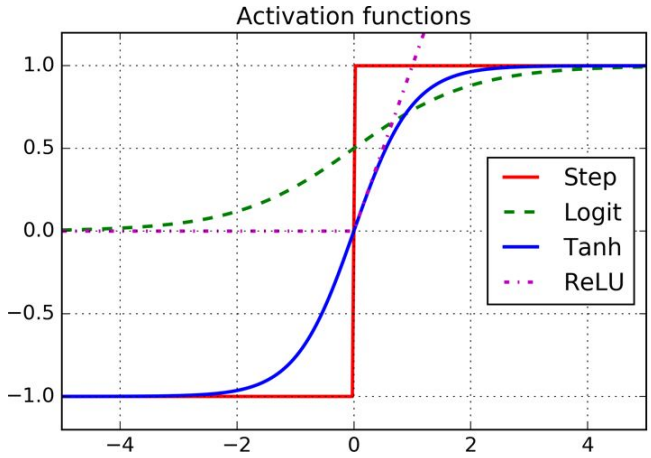
* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

Noção Geral Forward pass/Backpropagation

INSTITUTO FEDERAL
Goiás

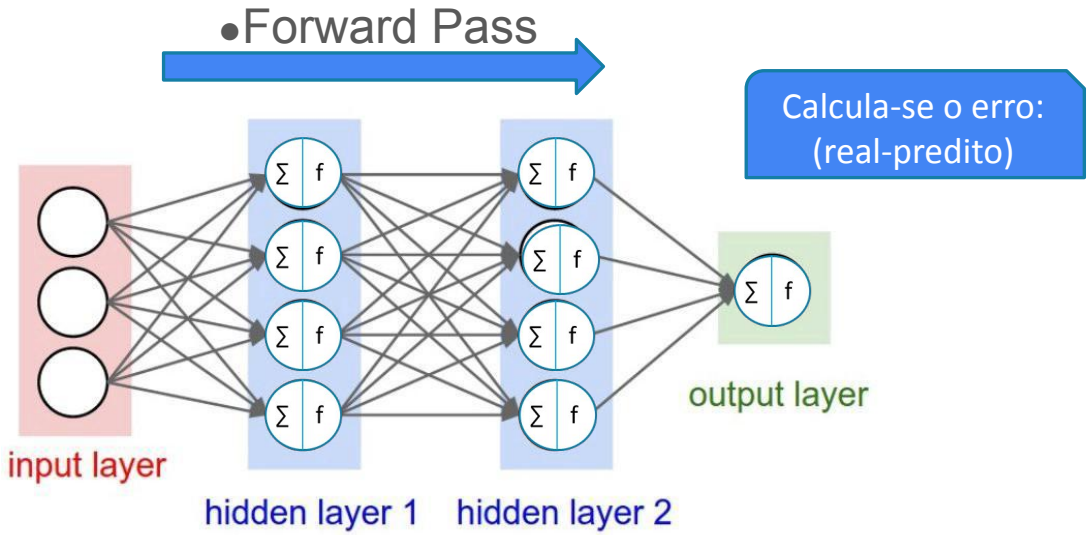
- Considere uma parte do treino (mini-batch), 32 instâncias por exemplo.
- Substituição da *step function* por *sigmoid function*.





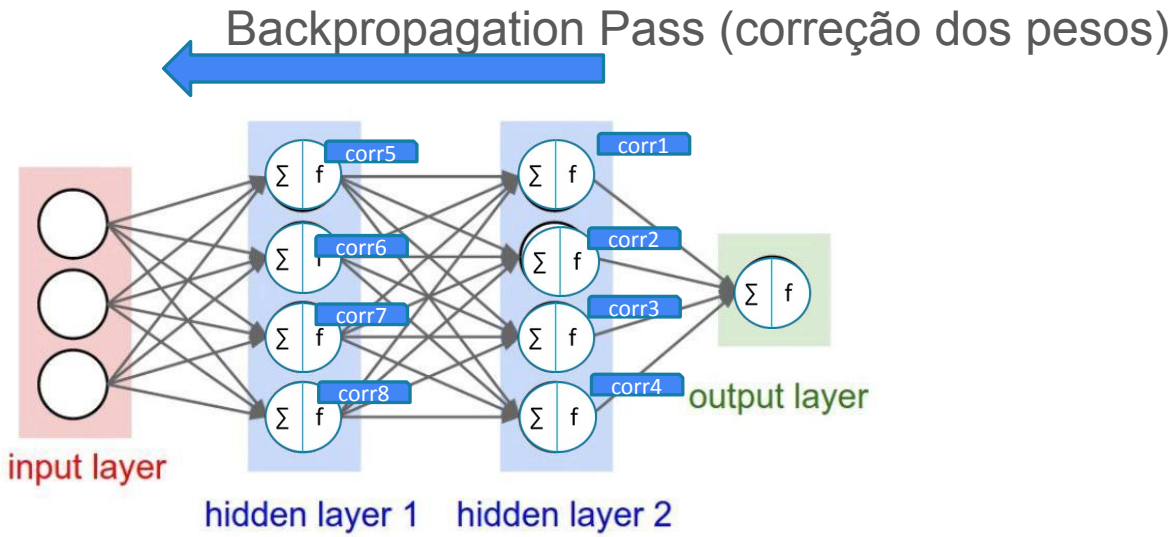
Noção Geral do MLP

- Considere uma parte do treino (mini-batch), 32 instâncias por exemplo.



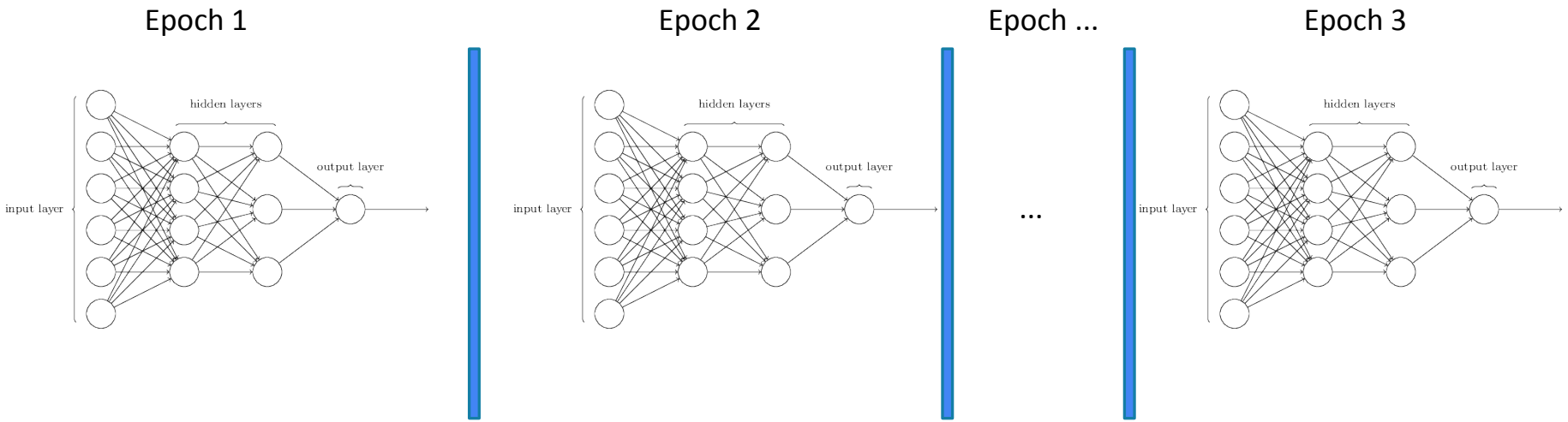
Noção Geral do MLP

- Considere uma parte do treino (mini-batch), 32 instâncias por exemplo.



Tudo na sua época...

- Em cada epoch
  - passada por toda coleção de treinamento (várias iterações)
    - A atualização dos pesos pode ser dividida em subconjuntos (batches) em cada época.



Backpropagation - Exemplo (propagação)

Tabela: Padrões

Entradas		Saida desejada	
x_1	x_2	y_1	y_2
0,05	0,1	0,01	0,99

Precisamos treinar a rede neural para fornecer as saidas desejadas

Relembrando a última aula: Elementos dos neurônios

- Entradas: x_1, x_2
- Saidas: y_1, y_2
- pesos sinápticos: $w_1, w_2, \ldots, w_8$ (Precisamos determinar por meio do treinamento)
- Funções de ativaçes de cada neurônio (escolha experimental)

Backpropagation - Exemplo (propagação)

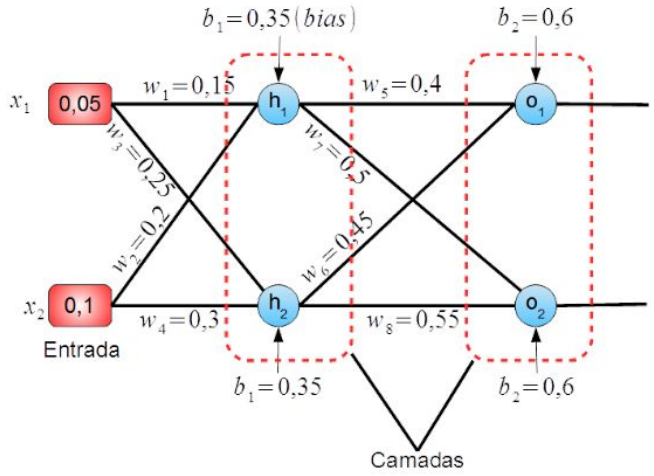
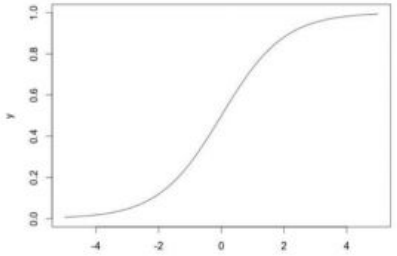


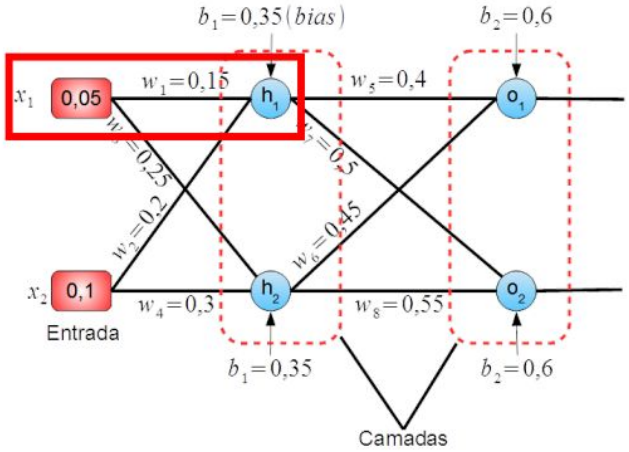
Figura: Rede Neural com pesos inicializados randomicamente.

$$\phi(x) = \frac{1}{1 + e^{-x}}$$

Parametrizando…: Usaremos em todos os neurônios a função de ativação sigmóide.



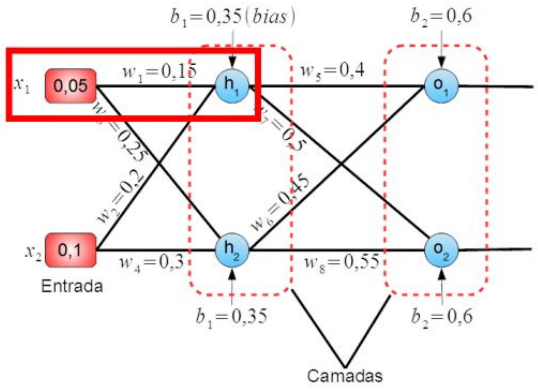
Backpropagation - Exemplo (propagação)



Propagando a entrada na primeira camada

- $u_{h1} = x_1 * w_1 + x_2 * w_2 + b_1 * 1$
- $u_{h1} = \boxed{0,05 * 0,15} + \boxed{0,1 * 0,2} + \boxed{0.35 * 1} = 0,3775$

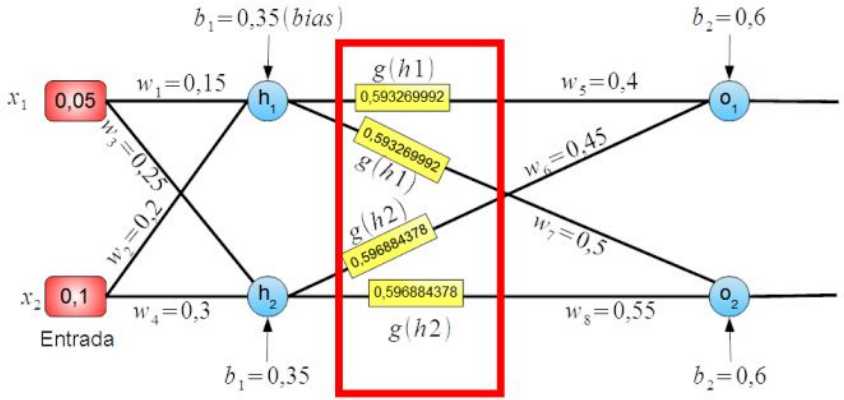
Backpropagation - Exemplo (propagação)



Propagando a entrada na primeira camada

- $u_{h1} = x_1 * w_1 + x_2 * w_2 + b_1 * 1$
- $u_{h1} = \boxed{0,05 * 0,15} + \boxed{0,1 * 0,2} + \boxed{0.35 * 1} = 0,3775$
- $u_{h2} = x_1 * w_3 + x_2 * w_4 + b_1 * 1$
- $u_{h2} = \boxed{0,05 * 0,25} + \boxed{0,1 * 0,3} + \boxed{0,35 * 1} = 0,39250$

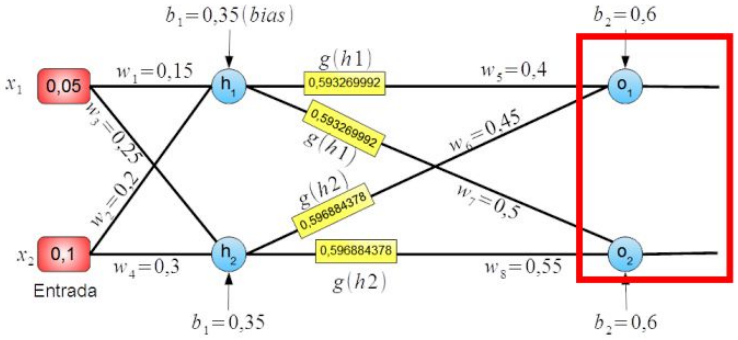
Backpropagation - Exemplo (propagação)



Propagando a entrada na primeira camada - função de ativação

- $g(h1) = g(u_{h1}) = \dfrac{1}{1+e^{u_{h1}}} = \dfrac{1}{1+e^{-0,3775}} = \boxed{0,593269992}$
- $g(h2) = g(u_{h2}) = \dfrac{1}{1+e^{u_{h2}}} = \dfrac{1}{1+e^{-0,3925}} = \boxed{0,596884378}$

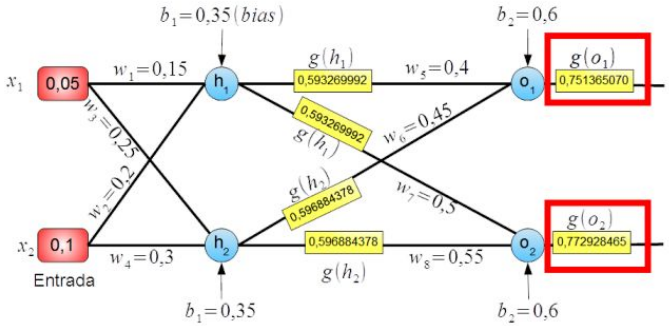
Backpropagation - Exemplo (propagação)



Propagando a entrada na segunda camada

- $u_{o1} = g(h1) * w_5 + g(h2) * w6 + b2 * 1 =$
  $0,593269992 * 0,4 + 0,596884378 * 0,45$+0,6*1 = 1,105905967
- $u_{o2} = g(h1) * w_7 + g(h2) * w_8 + b_2 * 1 =$
  $0,593269992 * 0,5 + 0,596884378 * 0,55 + 0,6 * 1 = 1,224921404$
- $g(o1) = g(u_{o1}) = \dfrac{1}{1 + e^{u_{o1}}} = \dfrac{1}{1 + e^{-1,105905967}} = \boxed{0,751365070}$
- $g(o2) = g(u_{o2}) = \dfrac{1}{1 + e^{u_{o2}}} = \dfrac{1}{1 + e^{-1,224921404}} = \boxed{0,772928465}$

Backpropagation - Exemplo (propagação)



Propagando a entrada na segunda camada

- $u_{o1} = g(h1) * w_5 + g(h2) * w6 + b2 * 1 =$
  $0,593269992 * 0,4 + 0,596884378 * 0,45$+0,6*1 = 1,105905967
- $u_{o2} = g(h1) * w_7 + g(h2) * w_8 + b_2 * 1 =$
  $0,593269992 * 0,5 + 0,596884378 * 0,55 + 0,6 * 1 = 1,224921404$
- $g(o1) = g(u_{o1}) = \frac{1}{1+e^{u_{o1}}} = \frac{1}{1+e^{-1,105905967}} = \boxed{0,751365070}$
- $g(o2) = g(u_{o2}) = \frac{1}{1+e^{u_{o2}}} = \frac{1}{1+e^{-1,224921404}} = \boxed{0,772928465}$

Backpropagation - Exemplo (retropropagação)

- Sabendo do erro, precisamos corrigir os pesos
  - Idéia do gradiente para correção.
- Os pesos são interdependentes
  - Funções que dependem do resultado outras funções (funções compostas)
- Conceitos importantes:
  - Derivadas/derivadas parciais
  - A regra da cadeia (chain rule) é usada quando temos uma composição de duas funções.

Backpropagation - Exemplo (retropropagação)

Derivadas…

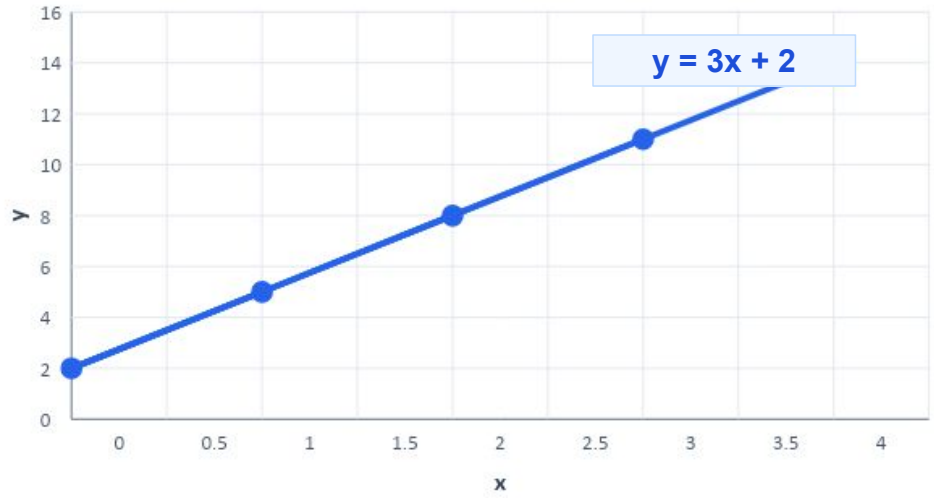
$$f(x) = (3x - 5x^2)^7$$

$$u = 3x - 5x^2 \rightarrow \frac{du}{dx} = 3 - 10x$$

$$y = u^7 \rightarrow \frac{dy}{du} = 7u^6$$

A derivada mede quanto y varia quando x varia

Em uma função linear, por exemplo, essa taxa de variação é constante em todos os pontos.



De x = 1 para x = 2: Δx = 1 e Δy = 3

Taxa de variação = Δy / Δx = 3 / 1 = 3

Função linear

$$f(x) = 3x + 2$$

O coeficiente 3 é a inclinação da reta. Ele indica que, para cada aumento de 1 em x, y aumenta 3.

DERIVADA

$$f'(x) = 3$$

Em todos os pontos da reta, a derivada vale 3.

Para nós a derivada representa a **taxa de variação de y em relação a x** em um ponto.

Backpropagation - Exemplo (retropropagação)

- Vamos compreender a regra da cadeia…
- A regra da cadeia (chain rule) é usada quando temos uma composição de duas funções.

Se $y = f(u)$ e $u = g(x)$ então $y = f(g(x))$

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

$$\frac{dy}{dx} = \frac{d}{dx}(f(g(x))) = f'(g(x))g'(x)$$

Backpropagation - Exemplo (retropropagação)

Sendo as derivadas…

$$f(x) = (3x - 5x^2)^7$$
$$u = 3x - 5x^2 \rightarrow \frac{du}{dx} = 3 - 10x$$
$$y = u^7 \rightarrow \frac{dy}{du} = 7u^6$$

Aplicando a regra da cadeia, temos:

Se $y = f(u)$ e $u = g(x)$ então $y = f(g(x))$

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$
$$\frac{dy}{dx} = 7u^6(3 - 10x) \quad \frac{dy}{dx} = 7(3x - 2x^2)^6(3 - 10x)$$

Backpropagation - Exemplo (retropropagação)

Primeiro, precisamos quantificar o quanto a rede errou - diferença entre a saída obtida e a desejada $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \tilde{y}_i)^2$

- $E_{total} = \frac{1}{2}\sum_{k=1}^{N}(d_k - g(ok))^2 = E_{o1} + E_{o2} =$

  $\frac{1}{2}\sum_{k=1}^{N}(y_1 - g(o1))^2 + \frac{1}{2}\sum_{k=1}^{N}(y_2 - g(o2))^2$

- $E_{o1} = \frac{1}{2}(y_1 - g(o1))^2 = \frac{1}{2} * (0,01 - 0,751365070)^2 = \boxed{0,274811083}$

Backpropagation - Exemplo (retropropagação)

Primeiro, precisamos quantificar o quanto a rede errou - diferença entre a saída obtida e a desejada

- $E_{total} = \frac{1}{2}\sum_{k=1}^{N}(d_k - g(ok))^2 = E_{o1} + E_{o2} =$

  $\frac{1}{2}\sum_{k=1}^{N}(y_1 - g(o1))^2 + \frac{1}{2}\sum_{k=1}^{N}(y_2 - g(o2))^2$

- $E_{o1} = \frac{1}{2}(y_1 - g(o1))^2 = \frac{1}{2} * (0,01 - 0,751365070)^2 = \boxed{0,274811083}$
- $E_{o2} = \frac{1}{2}(y_2 - g(o2))^2\frac{1}{2} * (0,99 - 0,772928465)^2 = \boxed{0,023560026}$

Backpropagation - Exemplo (retropropagação)

Primeiro, precisamos quantificar o quanto a rede errou - diferença entre a saída obtida e a desejada

- $E_{total} = \frac{1}{2}\sum_{k=1}^{N}(d_k - g(ok))^2 = E_{o1} + E_{o2} =$

  $\frac{1}{2}\sum_{k=1}^{N}(y_1 - g(o1))^2 + \frac{1}{2}\sum_{k=1}^{N}(y_2 - g(o2))^2$

- $E_{o1} = \frac{1}{2}(y_1 - g(o1))^2 = \frac{1}{2} * (0,01 - 0,751365070)^2 = \boxed{0,274811083}$
- $E_{o2} = \frac{1}{2}(y_2 - g(o2))^2 \frac{1}{2} * (0,99 - 0,772928465)^2 = \boxed{0,023560026}$
- $E_{total} = 0,274811083 + 0,023560026 = \boxed{0,298371109}$

Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

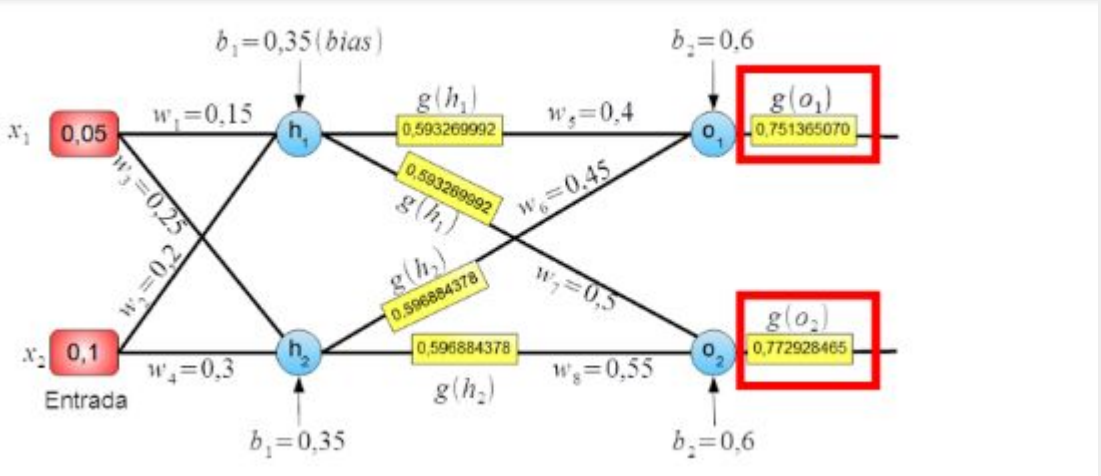
- $\frac{\partial E_{total}}{\partial w_5}$: Gradiente com respeito a w_5

Relembrando a notação:

$$E_{total} = E_{o1} + E_{o2} = \frac{1}{2}(y_1 - g(o1))^2 + \frac{1}{2}(y_2 - g(o2))^2$$

$$g(o1) = g(u_{o1}) = \frac{1}{1 + e^{u_{o1}}}$$

$$u_{o1} = g(h1) * w_5 + g(h2) * w6 + b2 * 1$$



Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

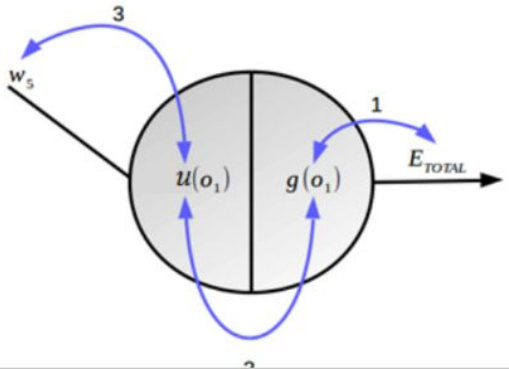
- $\frac{\partial E_{total}}{\partial w_5}$: Gradiente com respeito a w_5
- Segundo a regra da cadeia: $\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial w_5}$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$$

Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

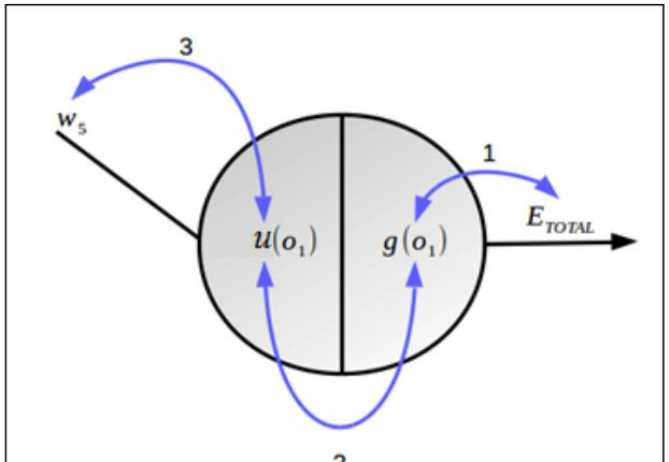
- $\frac{\partial E_{total}}{\partial w_5}$: Gradiente com respeito a w_5
- Segundo a regra da cadeia: $\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$



Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial w_5}$: Gradiente com respeito a w_5
- $\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$

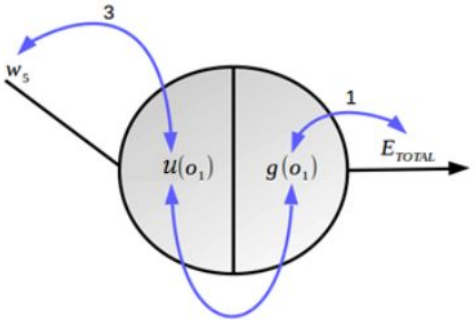


O ponto principal na retropropagação é “desmantelar” ou “quebrar” o processo usando a regra da cadeia para torná-lo o mais modular possível e usar a descida do gradiente para atualizar os pesos.

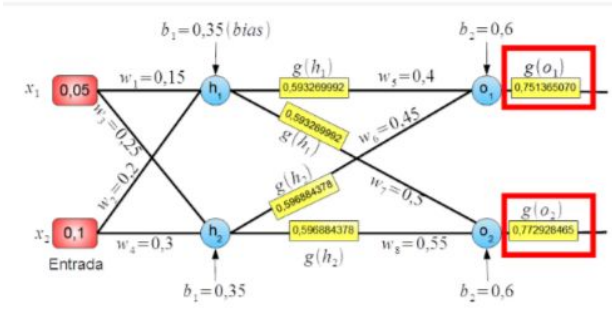
Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

- $E_{total} = \frac{1}{2} * (y_1 - g_{o1})^2 + \frac{1}{2} * (y_2 - g_{o2})^2$
- $\frac{\partial E_{total}}{\partial g_{o1}} = 2 * \frac{1}{2}(y_{o1} - g_{o1})^{2-1} * -1 + 0$(Constante na derivada em relação a g_{o1})
- $\frac{\partial E_{total}}{\partial g_{o1}} = -(y_1 - g_{o1}) = 0,01 - 0,75136507 = \boxed{0,74136507}$



$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$$



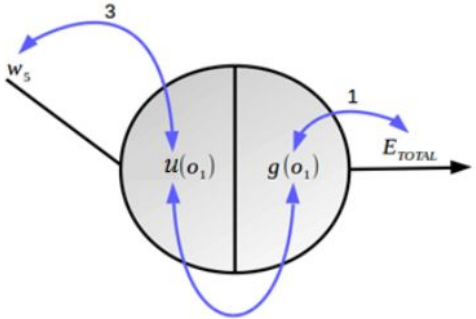
Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

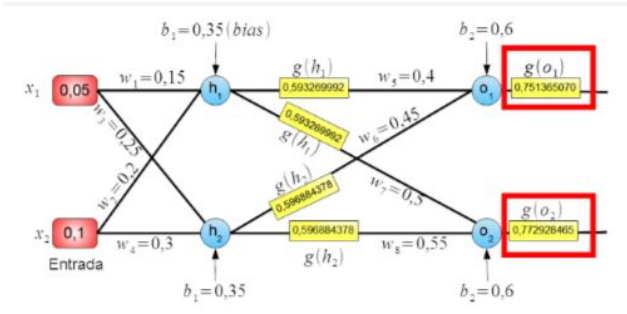
Agora a parcela $\frac{\partial g_{o1}}{\partial u_{o1}}$

https://builtin.com/articles/derivative-of-sigmoid-function

- $g_{o1} = \frac{1}{1+e^{-u_{o1}}}$ =saída da função de ativação
- $\frac{\partial g_{o1}}{\partial u_{o1}} = g_{o1}(1-g_{o1}) = 0,75136507 * (1-0,75136507) =$ $\boxed{0,186815602}$



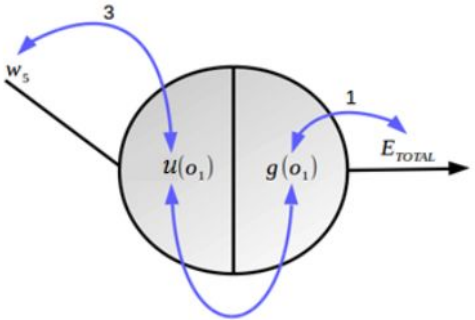
$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$$



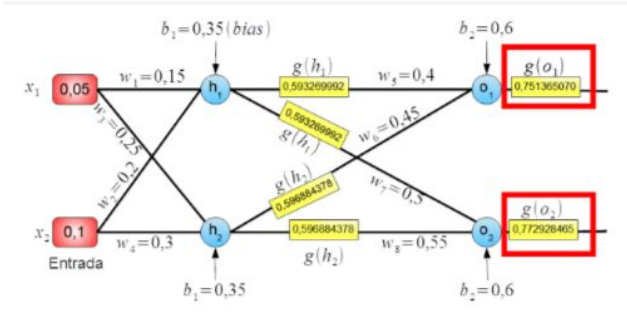
Backpropagation - Exemplo (retropropagação)

Agora a parcela $\frac{\partial u_{o1}}{\partial w_5}$

- $u_{o1} = w_5 * g_{h1} + w_6 * g_{h2} + b_2 * 1$
- $\frac{\partial u_{o1}}{\partial w_5} = 1 * g_{h1} * w_5^{1-1} + 0 + 0 = g_{h1} = \boxed{0,59326992}$



$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$$

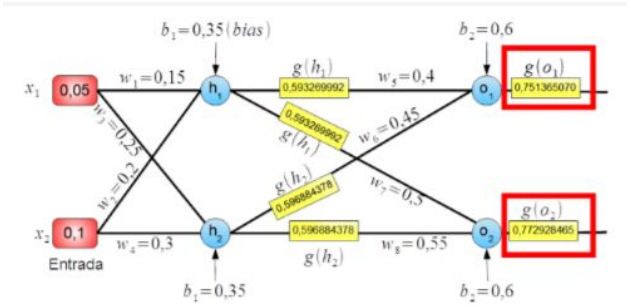


Backpropagation - Exemplo (retropropagação)

Corrigindo w_5: queremos estimar o quanto w_5 afeta o erro total (E)

Finalmente nós temos todas as parcelas

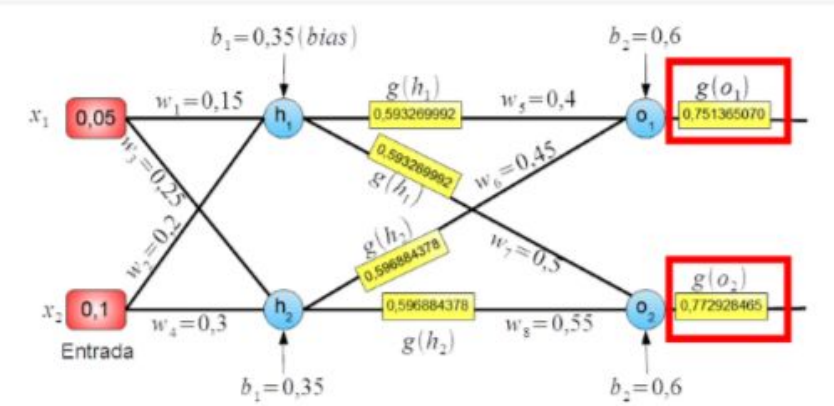
- $$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial w_5}$$
- $$\frac{\partial E_{total}}{\partial w_5} = 0,74136507 * 0,186815602 * 0,593269992 = 0,0822167041$$



Backpropagation - Exemplo (retropropagação)

Logo, considerando a taxa de aprendizado n=0.5:

$$w_5(t+1) = w_5(t) - \eta \frac{\partial E_{total}}{\partial w_5} = 0{,}4 - 0{,}5 * 0{,}0822167041 = 0{,}35891648$$



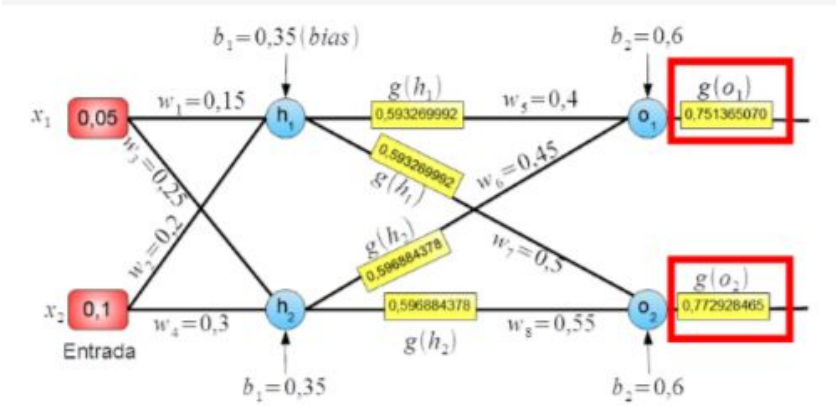
Backpropagation - Exemplo (retropropagação)

Mesma estratégia para atualizar os outros pesos sinápticos da última camada.

$$w_6(t+1) = 0,408666186$$

$$w_7(t+1) = 0,511301270$$

$$w_8(t+1) = 0,561370121$$

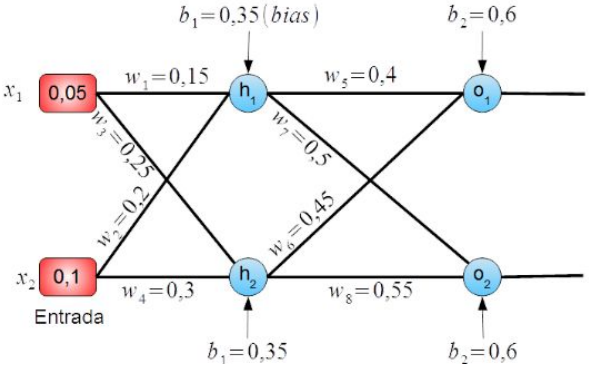


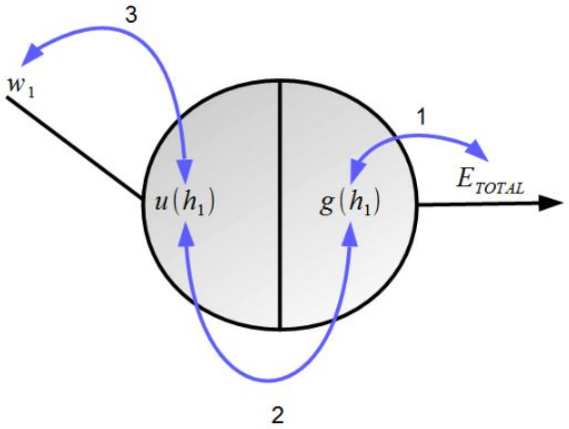
Backpropagation - Exemplo (retropropagação)

Agora, vamos atualizar os pesos da primeira camada.

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial w_1}$: Gradiente com respeito a w_1
- $\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial g_{h1}} * \frac{\partial g_{h1}}{\partial u_{h1}} * \frac{\partial u_{h1}}{\partial w_1}$

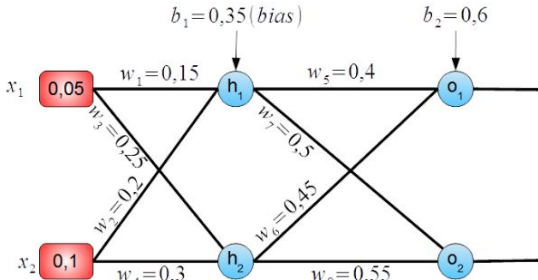




Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial w_1}$: Gradiente com respeito a w_1
- $\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial g_{h1}} * \frac{\partial g_{h1}}{\partial u_{h1}} * \frac{\partial u_{h1}}{\partial w_1}$
- Temos três parcelas para resolver a) $\frac{\partial E_{total}}{\partial g_{h1}}$, b) $\frac{g_{h1}}{\partial u_{h1}}$ e, por fim, c) $\frac{\partial u_{h1}}{\partial w_1}$
- Para resolver a), temos que lembrar que $E_{total} = E_{o1} + E_{o2}$
- $E_{o1} = \frac{1}{2} * (d_1 - g_{o1})^2$ e $E_{o2} = \frac{1}{2} * (d_2 - g_{o2})^2$
- $\frac{\partial E_{total}}{\partial g_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial g_{h1}}} + \boxed{\frac{\partial E_{o2}}{\partial g_{h1}}}$, não temos essas parcelas

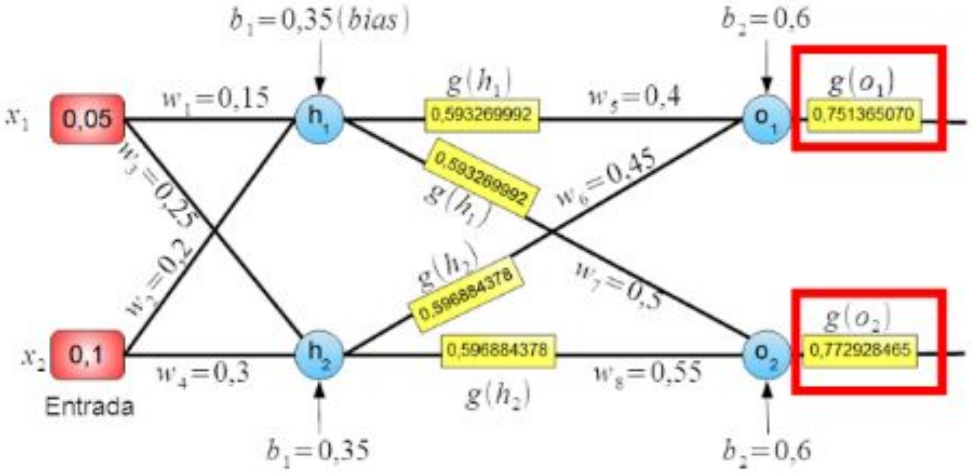


Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial g_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial g_{h1}}} + \boxed{\frac{\partial E_{o2}}{\partial g_{h1}}}$, não temos essas parcelas :(

… mas sabemos que a saída saída do erro depende de uo1, e a saída de uo1 depende de gh1

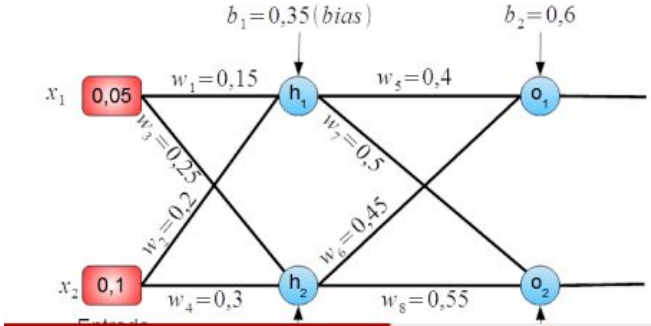


Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial g_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial g_{h1}}} + \boxed{\frac{\partial E_{o2}}{\partial g_{h1}}}$, não temos essas parcelas :(
- $\frac{\partial E_{o1}}{\partial g_{h1}} = \frac{\partial E_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial g_{h1}}$ = ?
- $\frac{\partial E_{o1}}{\partial u_{o1}} = \frac{\partial E_{o1}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} = 0,74136507 * 0,186815602 = \boxed{0,138498562}$

Valores calculados anteriormente
(a derivada do erro em relação à saída g_{o1} e entrada u_{o1} do neurônio de saída o_1.)

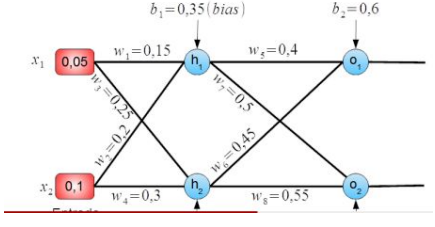


Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial g_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial g_{h1}}} + \boxed{\frac{\partial E_{o2}}{\partial g_{h1}}}$, não temos essas parcelas :(
- $\frac{\partial E_{o1}}{\partial g_{h1}} = \frac{\partial E_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial g_{h1}}$ = ?
- $\frac{\partial E_{o1}}{\partial u_{o1}} = \frac{\partial E_{o1}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} = 0,74136507 * 0,186815602 = \boxed{0,138498562}$
- $\frac{\partial u_{o1}}{\partial g_{h1}} = w_5$, pois, $u_{o1} = w_5 * g_{h1} + \cancel{w_6 * g_{h2} + b_2 * 1}$ =0,4

constante quando derivamos para g_{h1}



Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial g_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial g_{h1}}} + \boxed{\frac{\partial E_{o2}}{\partial g_{h1}}}$, não temos essas parcelas :(
- $\frac{\partial E_{o1}}{\partial g_{h1}} = \frac{\partial E_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial g_{h1}}$ = ?
- $\frac{\partial E_{o1}}{\partial u_{o1}} = \frac{\partial E_{o1}}{\partial g_{o1}} * \frac{\partial g_{o1}}{\partial u_{o1}} = 0,74136507 * 0,186815602 = \boxed{0,138498562}$
- $\frac{\partial u_{o1}}{\partial g_{h1}} = w_5$, pois, $u_{o1} = w_5 * g_{h1} + \boxed{\cancel{w_6 * g_{h2} + b_2 * 1}}$ =0,4
- $\frac{\partial E_{o1}}{\partial g_{h1}} = \frac{\partial E_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial g_{h1}} = 0,138498562 * 0,4 = 0,055399425$

Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- $\frac{\partial E_{total}}{\partial g_{h1}} = \boxed{\frac{\partial E_{o1}}{\partial g_{h1}}} + \boxed{\frac{\partial E_{o2}}{\partial g_{h1}}}$, não temos essas parcelas :(

- $\frac{\partial E_{o1}}{\partial g_{h1}} = \frac{\partial E_{o1}}{\partial u_{o1}} * \frac{\partial u_{o1}}{\partial g_{h1}} = 0,138498562 * 0,4 = 0,055399425$
- Usando os mesmos passos, $\frac{\partial E_{o2}}{\partial g_{h1}} = -0,019049119$
- Logo, $\frac{\partial E_{Total}}{\partial g_h 1} = 0,055399425 - 0,019049119 = \boxed{0,036350306}$

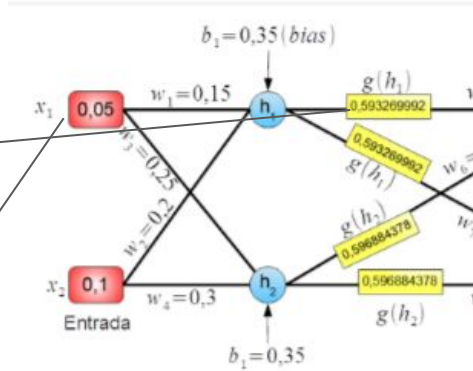
- $\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial g_{h1}} * \frac{\partial g_{h1}}{\partial u_{h1}} * \frac{\partial u_{h1}}{\partial w_1}$

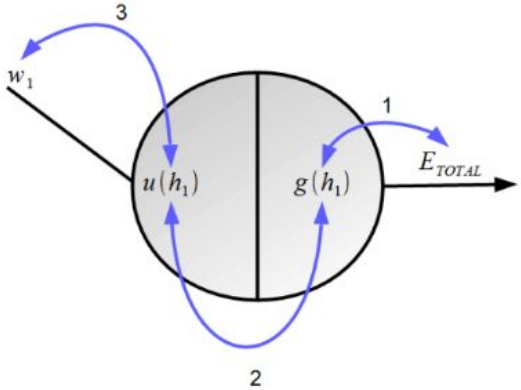
ok

Backpropagation - Exemplo (retropropagação)

Corrigindo w_1: queremos estimar o quanto w_1 afeta o erro total (E)

- Agora falta: $\frac{\partial g_{h1}}{\partial u_{h1}}$, sabemos que: $g_{h1} = \frac{1}{1+e^{-u_{h1}}}$
- $\frac{\partial g_{h1}}{\partial u_{h1}} = g_{h1} * (1 - g_{h1}) = 0,59326999 * (1 - 0,59326999) =$ $0,241300709$
- Falta, $\frac{\partial u_{h1}}{\partial w_1}$, sabemos que $u_{h1} = w_1 * x_1 + w_2 * x_2 + b_1 * 1$
- Derivando em relação a w_1, temos que, $\frac{\partial u_{h1}}{\partial w_1} = x_1 =$ $0,05$
- Finalmente:
- $\frac{\partial E_{total}}{\partial w_1} = 0,036350306 * 0,241300709 * 0,05 =$ $0,000438568$
- $w_1(t+1) = w_1(t) - \eta * \frac{\partial E_{total}}{\partial w_1}$





Backpropagation - Exemplo (retropropagação)

- Atualizando w_1, $w_1(t+1) = w_1(t) - \eta * \frac{\partial E_{total}}{\partial w_1}$
- $w_2(t+1) = 0,19956143$
- $w_3(t+1) = 0,24975114$
- $w_4(t+1) = 0,29950229$
- Com os pesos sinápticos no instante t, $E = 0,298371109$
- Com os pesos sinápticos no instante $t+1$, $E = 0,291027924$
- Com os pesos sinápticos no instante $t+10000$, $E = 0,000035085$
- Saídas calculadas no instante $t+10000$,
- Para $(y_1 = 0,01)$ temos $0,015912196$
- Para $(y_2 = 0,99)$ temos $0,984065734$

Backpropagation - Considerações

- Online (stochastic) learning vs (full-)Batches vs Mini-Batches
- Para o pseudo-código abaixo, como devemos tratar os múltiplos exemplos de treino?

Predição(w, x, bias)

ativação = sigmoid(w^T * x + bias)

retorna ativação

Repita (até convergência dos pesos)

Para cada instância (x,y)

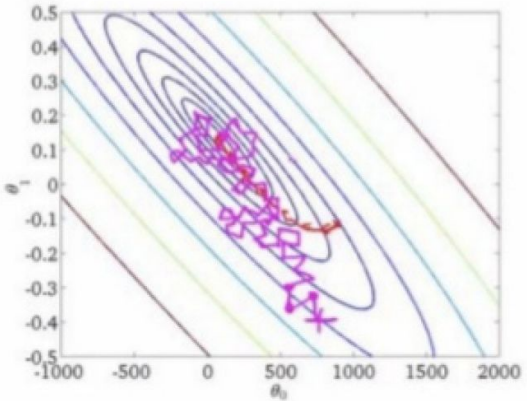
bias=bias - α *gradiente_bias

Para cada peso w_k

w[k] = **w**[k] - α**gradiente*_x[k]

Backpropagation - Considerações

- Online (stochastic) learning vs Batches vs Mini-Batches
- Online (stochastic) learning:
  - Para cada exemplo, calcula o erro desse exemplo, atualiza o peso da rede com o gradiente desse único exemplo.
  - Problemas:
    - Um único exemplo pode alterar muito os pesos a cada iteração, “perdendo o rumo”!
    - Convergência mais demorada e mais instável!



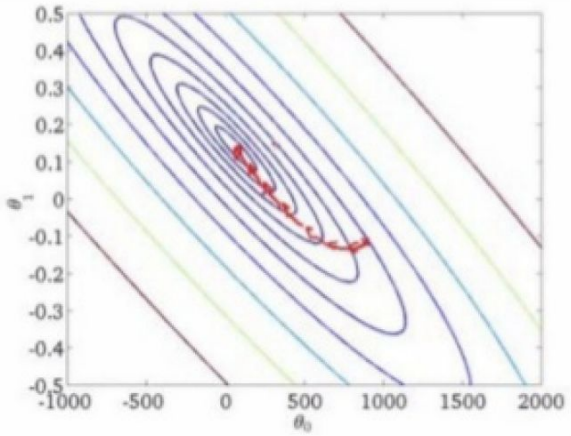
Stochastic: single-example gradient

Entretanto, segundo Goodfellow, com stochastic learning adicionamos mais “aleatoriedade” à busca, e portanto, menor chance de overfitting com learning rate baixo.

Fonte: https://papyrus.bib.umontreal.ca/xmlui/bitstream/handle/1866/21287/Zhang_Ying_2018_memoire.pdf?sequence=4&isAllowed=y

Backpropagation - Considerações

- Online (stochastic) learning vs Batch vs Mini-Batches
- Batch learning:
  - Para cada exemplo, calcula o erro desse exemplo…
    - mas atualiza o cada peso da rede com o gradiente médio de todos os exemplos!
  - Vantagem: O modelo caminha para de forma mais consistente/rápida para um ótimo local
  - Desvantagem: Usar todos os exemplos de uma vez “foca” muito o modelo para uma região do espaço



Batch: gradient

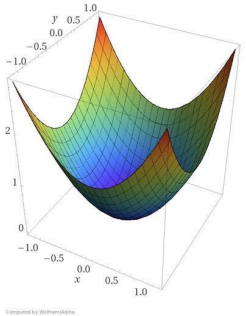
Fonte: https://papyrus.bib.umontreal.ca/xmlui/bitstream/handle/1866/21287/Zhang_Ying_2018_memoire.pdf?sequence=4&isAllowed=y

Backpropagation - Considerações

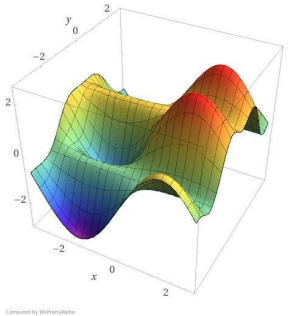
Importância dos mini-batches:

- Cada batch é um problema de otimização diferente:
- Portanto, temos menos chance de nos prendermos aos "mínimos locais"
- Entretanto, mini-batches pequenos alteram muito o problema de otimização entre as interações do aprendizado.

ex. batch 1:



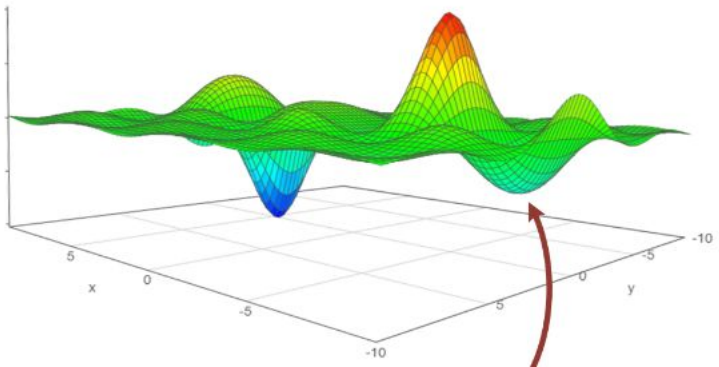
batch 2….



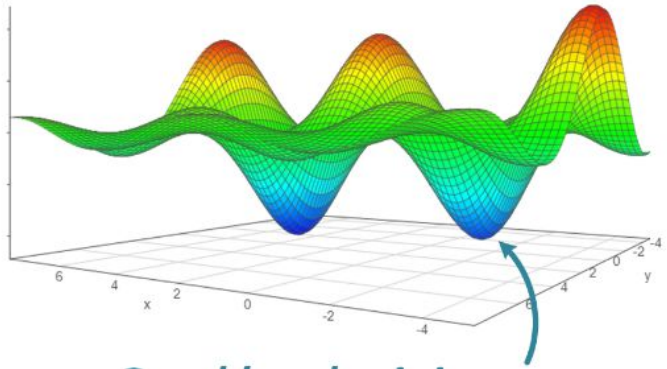
Backpropagation - Considerações

- Online (stochastic) learning vs Batches vs Mini-Batches
- Mini-Batch:
  - Meio-termo entre online e batch
    - Atualiza o cada peso da rede com o gradiente médio de N exemplos!
  - Vantagem: dependendo de N, percorre melhor o espaço de forma eficiente
  - Desvantagem: Parâmetro N
- Ideal na prática!

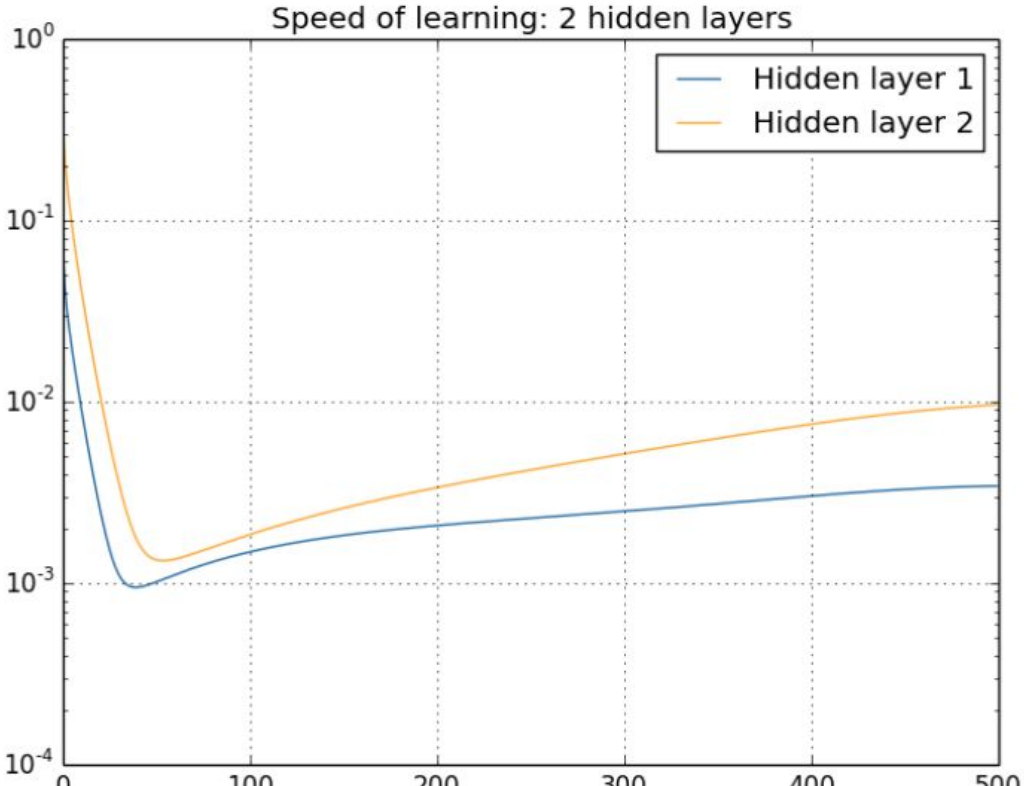
Loss com exemplos de Treino



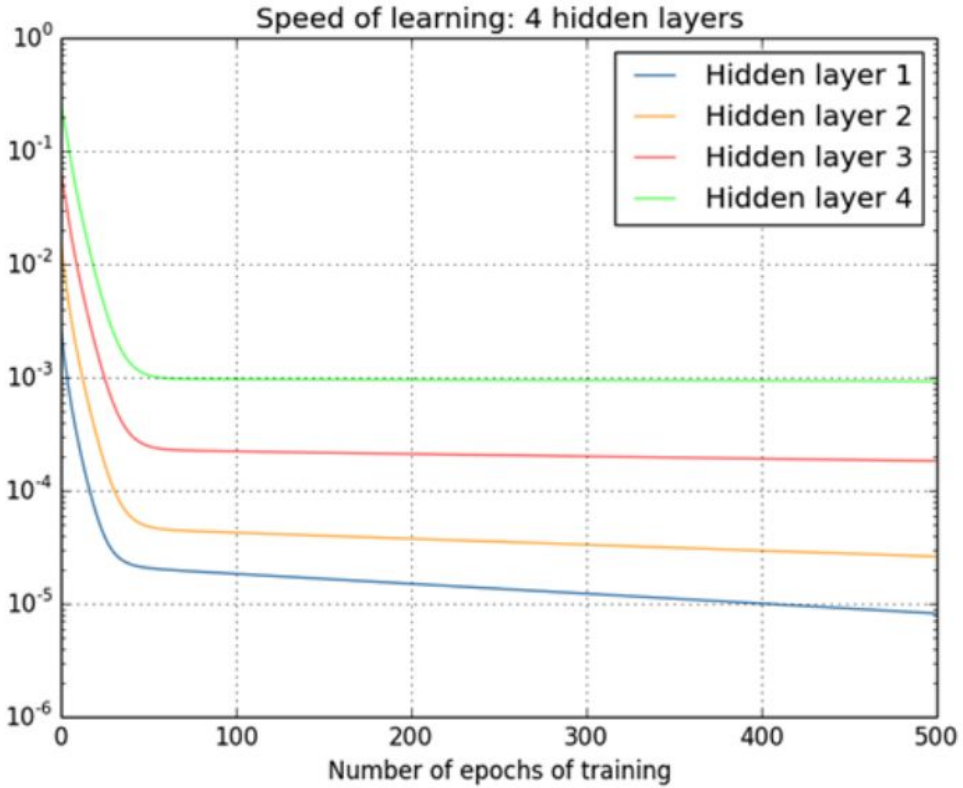
Loss com exemplos de Teste (desconhecido)

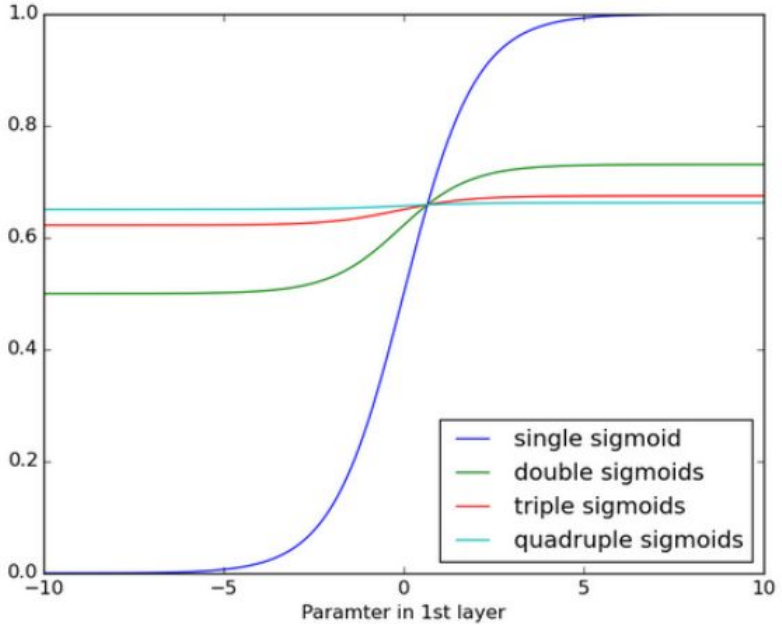


Backpropagation - Considerações



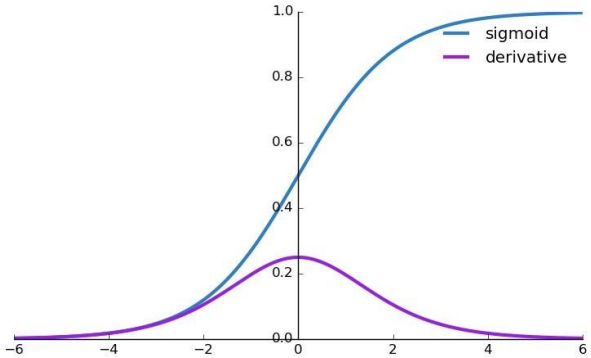
Backpropagation - Considerações

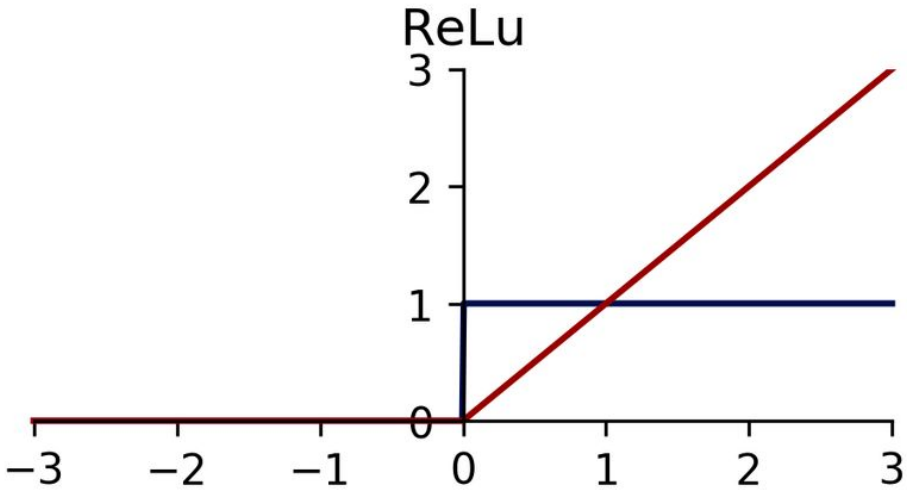




https://blog.csdn.net/omnispace/article/details/78151053

Backpropagation - Considerações





Keras + TF2

- https://Keras.io
  - High-level Deep Learnig API
- Keras utiliza-se de bibliotecas DL como (multibackend keras):
  - TensorFlow (Keras é sua API oficial)
  - Microsoft Cognitive Toolkit – CNTK
  - Theano
  - ...
- Concorrente do Keras
  - Facebook's PyTorch
- Keras e PyTorch são inpirados no Scikit-Learn

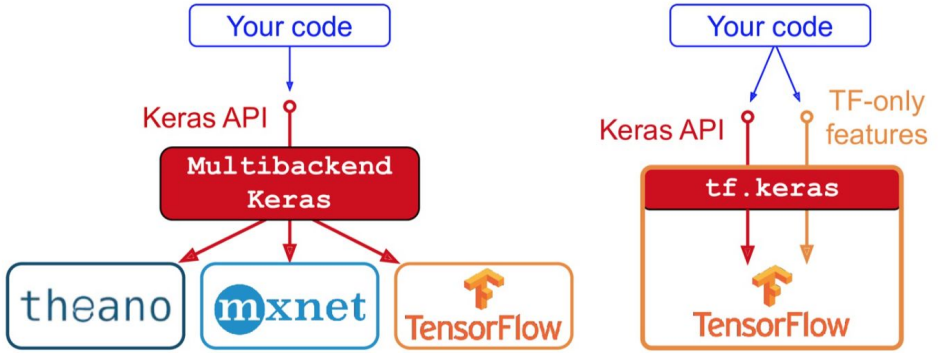


Figure 10-10. Two implementations of the Keras API: multibackend Keras (left) and tf.keras (right)

Instalação

- Depois de instalar
  - Jupyter
  - Scikit-Learn
- python3 pip install – upgrade tensorflow
- Teste tudo com:

```
In [9]: import tensorflow as tf
        from tensorflow import keras

In [10]: tf.__version__
Out[10]: '2.1.0'

In [11]: keras.__version__
Out[11]: '2.2.4-tf'
```

link do colab:
https://colab.research.google.com/drive/1LSKFJCPKQATwEknZo-SE2L5-BdJq5rF8?usp=sharing

In [1]:
```
from tensorflow import keras
```

Base de Dados usada Fashion MNIST. 70.000 imagens em escala de cinza, com 784 features (28x28). Keras disponibiliza diversas bases de dados para tratamento. ¶

In [2]:
```
fashion_mnist = keras.datasets.fashion_mnist
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist.load_data()
```

Diferente do Scikit-Learn, as imagens estão no formato 28x28 e não em uma dimensão. Ainda, o tipo de dado é inteiro.

In [3]:
```
print (X_train_full.shape, X_train_full.dtype)
```

```
(60000, 28, 28) uint8
```

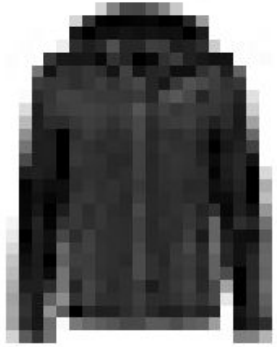
Embora o treino esteja dividido em treino e teste, vamos ter que criar o conjunto de validação.

Outra coisa importante é converter os dados para escala 0-1

In [4]:
```
X_valid, X_train = X_train_full[:5000] / 255., X_train_full[5000:] / 255.
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
X_test = X_test / 255.
```

Vamos ver uma instância:

In [5]:
```
import matplotlib.pyplot as plt
plt.imshow(X_train[0], cmap="binary")
plt.axis('off')
plt.show()
```



In [6]:
```
y_train
```

Out[6]:
```
array([4, 0, 7, ..., 3, 0, 5], dtype=uint8)
```

Nesse caso, cada classe se refere a uma peça de roupa

In [7]:
```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

In [8]:
```
class_names[y_train[0]]
```

Out[8]:
```
'Coat'
```

Vamos ver uma amostra das instâncias, 40 delas...

In [9]:
```
n_rows = 10
n_cols = 10
plt.figure(figsize=(n_cols * 1.2, n_rows * 1.2))
for row in range(n_rows):
    for col in range(n_cols):
        index = n_cols * row + col
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(X_train[index], cmap="binary", interpolation="nearest")
        plt.axis('off')
        plt.title(class_names[y_train[index]], fontsize=12)
plt.subplots_adjust(wspace=0.2, hspace=0.5)
save_fig('fashion_mnist_plot', tight_layout=False)
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-9-15d96644bf85> in <module>
     10         plt.title(class_names[y_train[index]], fontsize=12)
     11 plt.subplots_adjust(wspace=0.2, hspace=0.5)
---> 12 save_fig('fashion_mnist_plot', tight_layout=False)
     13 plt.show()

NameError: name 'save_fig' is not defined
```



Vamos então construir uma Rede Neural

```
model = keras.models.Sequential()
model.add(keras.layers.Flatten(input_shape=[28, 28]))
model.add(keras.layers.Dense(300, activation="relu"))
model.add(keras.layers.Dense(100, activation="relu"))
model.add(keras.layers.Dense(10, activation="softmax"))
```

A primeira camada transforma a entrada em um vetor de uma dimensão. Essa primeira camada não tem pesos, somente as entradas

As demais camadas são densas pois atribuem os pesos para camada anterior e os neurônios da camada. Vejam o parâmetro para função de ativação, ReLU.

A camada densa tem sua matriz de pesos e o vetor de bias. Vejam que uma camada densa calcula para cada neurônio a função h(X)=FunçaoAtivaçao(XW+b)

A última camada possuem 10 neurônios pois estamos tratando de 10 classes exclusivas.

A função summary descreve a rede criada.

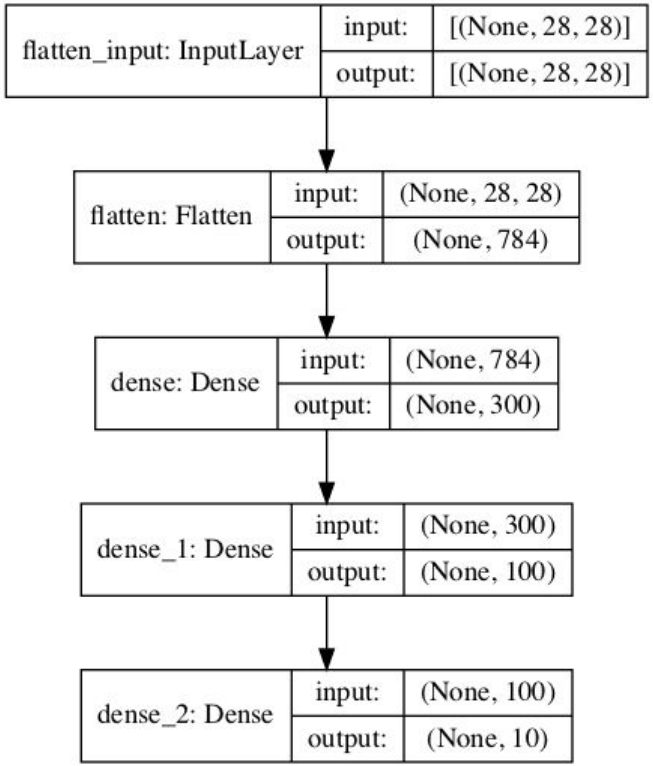
```
]: model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 300)               235500
_________________________________________________________________
dense_1 (Dense)              (None, 100)               30100
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1010
=================================================================
Total params: 266,610
Trainable params: 266,610
Non-trainable params: 0
_________________________________________________________________
```

Note que 23550 parâmetros se refere aos pesos e bias que serão aprendidos, 784x300 no caso da camda "Dense".

Uma representação gráfica dessa rede pode ser obtida:

In [22]:
```
keras.utils.plot_model(model, "my_fashion_mnist_model.png", show_shapes=True)
```



Os valores dos pesos e bias em cada camada podem ser acessados;

In [23]:
```
model.layers
```

Out[23]:
```
[<tensorflow.python.keras.layers.core.Flatten at 0x1685be25d08>,
 <tensorflow.python.keras.layers.core.Dense at 0x1685be25c08>,
 <tensorflow.python.keras.layers.core.Dense at 0x1685be54d88>,
 <tensorflow.python.keras.layers.core.Dense at 0x1685c077bc8>]
```

In [24]:
```
hidden1 = model.layers[1]
hidden1.name
```

Out[24]:
```
'dense'
```

In [25]:
```
weights, biases = hidden1.get_weights()
```

In [26]:
```
print ("Olá, eu sou o conhecimento:", weights)
```

```
Olá, eu sou o conhecimento: [[-0.01982554 -0.05972073 -0.07404873 ... -0.070458   -0.06915936
   0.06313017]
 [-0.01207474 -0.02712235 -0.06105417 ...  0.02781773 -0.04483516
   0.0124708 ]
 [ 0.05548631  0.03520936 -0.03441961 ...  0.03328292  0.02233611
   0.06562936]
 ...
 [-0.04838773  0.06220061  0.03437502 ...  0.02119237  0.05427265
   0.04171459]
 [-0.03504748 -0.01263107 -0.02968967 ...  0.02076242 -0.07128673
   0.02752175]
 [ 0.00569931  0.0725559   0.07225588 ... -0.06055948  0.02769873
   0.01649418]]
```

In [31]:
```
biases
```

Out[31]:
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
       
```

Compilando...

In [28]:
```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])
```

Rodando...

In [*]:
```
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid))
```

```
Epoch 1/30
1719/1719 [==============================] - 6s 4ms/step - loss: 0.7131 - accuracy: 0.7634 - val_loss: 0.5110 - val_accuracy:
0.8228
Epoch 2/30
1719/1719 [==============================] - 7s 4ms/step - loss: 0.4885 - accuracy: 0.8284 - val_loss: 0.4431 - val_accuracy:
0.8478
Epoch 3/30
1719/1719 [==============================] - 6s 4ms/step - loss: 0.4432 - accuracy: 0.8453 - val_loss: 0.4522 - val_accuracy:
0.8382
Epoch 4/30
1719/1719 [==============================] - 7s 4ms/step - loss: 0.4173 - accuracy: 0.8531 - val_loss: 0.3873 - val_accuracy:
0.8650
Epoch 5/30
```

```
Epoch 25/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2480 - accuracy: 0.9108 - val_loss: 0.3021 - val_accuracy:
0.8930
Epoch 26/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2441 - accuracy: 0.9122 - val_loss: 0.3201 - val_accuracy:
0.8836
Epoch 27/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2403 - accuracy: 0.9134 - val_loss: 0.2899 - val_accuracy:
0.8966
Epoch 28/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2354 - accuracy: 0.9150 - val_loss: 0.3014 - val_accuracy:
0.8926
Epoch 29/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2318 - accuracy: 0.9158 - val_loss: 0.3116 - val_accuracy:
0.8920
Epoch 30/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2279 - accuracy: 0.9182 - val_loss: 0.3153 - val_accuracy:
0.8846
```

Veja que accuracy e val_accuracy te ajuda a verificar o overfitting.

Em casos em que a Base de Dados está muito desbalanceada, é útil a utilização do class_weith

Em caso de instâncias que tenham sido rotuladas por pessoas, e merecem mais peso, é possível usar sample_weight.

O método fit retorna um objeto HISTORY com parâmetros de treinament (history.params), epchs executados (history.epoch), e loss e métrica avaliada. Vejam como podemos observar a execução da rede.

In [37]:
```
history.history.keys()
```

Out[37]:
```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

O método fit retorna um objeto HISTORY com parâmetros de treinament (history.params), epchs executados (history.epoch), e loss e métrica avaliada. Vejam como podemos observar a execução da rede.

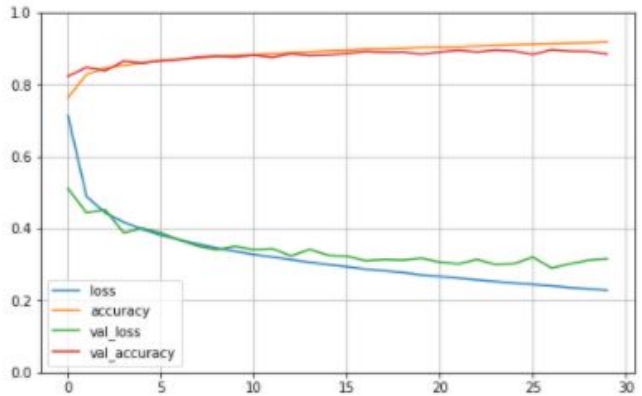
In [30]:
```
history.history.keys()
```

Out[30]:
```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

In [31]:
```
import pandas as pd
pd.DataFrame(history.history).plot(figsize=(8, 5))
plt.grid(True)
plt.gca().set_ylim(0, 1)
save_fig("keras_learning_curves_plot")
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-31-d3ce0e3ea8e7> in <module>
      3 plt.grid(True)
      4 plt.gca().set_ylim(0, 1)
----> 5 save_fig("keras_learning_curves_plot")
      6 plt.show()

NameError: name 'save_fig' is not defined
```



Após avaliar seu método com a validação, é possível avaliar no teste.

In [32]:
```
model.evaluate(X_test, y_test)
```

```
313/313 [==============================] - 1s 3ms/step - loss: 0.3556 - accuracy: 0.8707
```

Out[32]:
```
[0.3556255102157593, 0.8707000017166138]
```

Fazendo predição de 3 imagens

In [33]:
```
X_new = X_test[:3]
y_proba = model.predict(X_new)
y_proba.round(2)
```

Out[33]:
```
array([[0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.01, 0.  , 0.99],
       [0.  , 0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ],
       [0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ]],
      dtype=float32)
```

In [34]:
```
y_pred = model.predict_classes(X_new)
y_pred
```

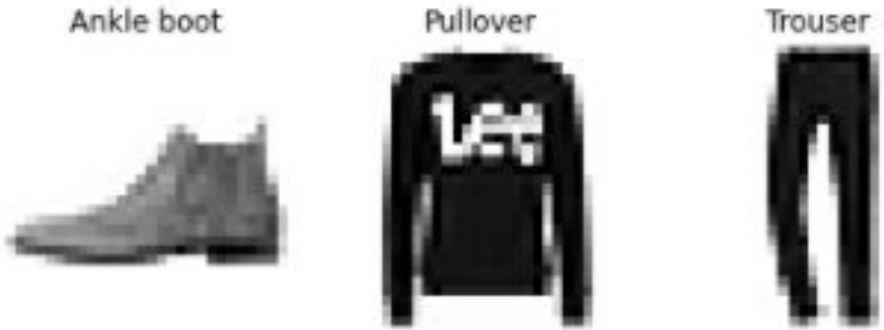
```
WARNING:tensorflow:From <ipython-input-34-81ace37e545f>:1: Sequential.predict_classes (from tensorflow.python.keras.engine.sequential) is deprecated and will be removed after 2021-01-01.
Instructions for updating:
Please use instead:* `np.argmax(model.predict(x), axis=-1)`,   if your model does multi-class classification   (e.g. if it uses a `softmax` last-layer activation).* `(model.predict(x) > 0.5).astype("int32")`,   if your model does binary classification   (e.g. if it uses a `sigmoid` last-layer activation).
```

Out[34]:
```
array([9, 2, 1], dtype=int64)
```

In [35]:
```
import numpy as np
np.array(class_names)[y_pred]
```

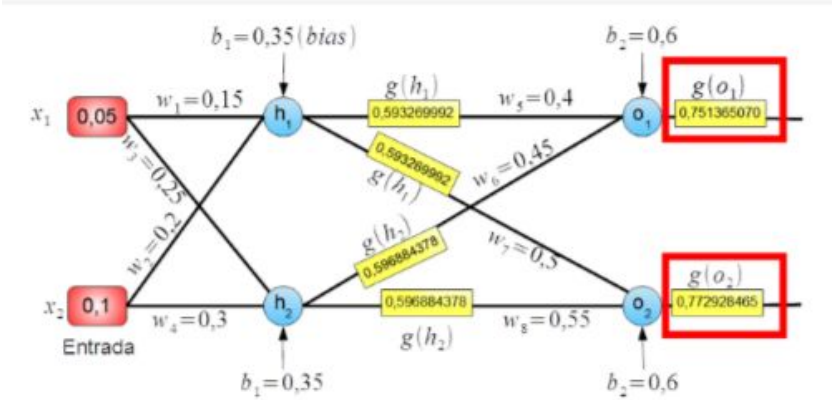
Out[35]:
```
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')
```

In [37]:
```
y_new = y_test[:3]
plt.figure(figsize=(7.2, 2.4))
for index, image in enumerate(X_new):
    plt.subplot(1, 3, index + 1)
    plt.imshow(image, cmap="binary", interpolation="nearest")
    plt.axis('off')
    plt.title(class_names[y_test[index]], fontsize=12)
plt.subplots_adjust(wspace=0.2, hspace=0.5)
plt.show()
```



Atividade

1) Execute o código do (disponibilizado no moodle). Corrija e descreva o problema do mesmo, apresentando uma estratégia para avaliar a evolução dos pesos nas camadas da rede.
2) Seguindo o exemplo da aula, apresente o cálculo do gradiente para o peso w7 (figura abaixo). Assuma as mesmas funções de erro e de ativação do exemplo.



Referências

[1] Aurélien Géron. Neural networks and deep learning https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html

[2] NIELSEN, M. A. Neural Networks and Deep Learning. Determination Press, 2015. [Online]. Disponível em <http://neuralnetworksanddeeplearning.com/>

[3] GOODFELLOW, Y. BENGIO, A. COURVILLE. Deep Learning. MIT Press, 2017.